A Systematic Literature Review on LLM Defenses Against Prompt Injection and Jailbreaking: Expanding NIST Taxonomy

Pedro H. Barcha Correia[a,*], Ryan W. Achjian[a], Diego E. G. Caetano de Oliveira[c], Ygor Acacio Maria[a], Victor Takashi Hayashi[a], Marcos Lopes[b], Charles Christian Miers[c], Marcos A. Simplicio Jr.[a]

[a] *Universidade de São Paulo (USP), Laboratório de Arquitetura e Redes de Computadores (LARC), Brazil*
[b] *Universidade de São Paulo (USP), Departamento de Linguística, Brazil*
[c] *Universidade do Estado de Santa Catarina (UDESC), Programa de Pós-Graduação em Computação Aplicada (PPGCAP), Brazil*

Abstract

The rapid advancement and widespread adoption of generative artificial intelligence (GenAI) and large language models (LLMs) has been accompanied by the emergence of new security vulnerabilities and challenges, such as jailbreaking and other prompt injection attacks. These maliciously crafted inputs can exploit LLMs, causing data leaks, unauthorized actions, or compromised outputs, for instance. As both offensive and defensive prompt injection techniques evolve quickly, a structured understanding of mitigation strategies becomes increasingly important. To address that, this work presents the first systematic literature review on prompt injection mitigation strategies, comprehending 88 studies. Building upon NIST's report on adversarial machine learning, this work contributes to the field through several avenues. First, it identifies studies beyond those documented in NIST's report and other academic reviews and surveys. Second, we propose an extension to NIST taxonomy by introducing additional categories of defenses. Third, by adopting NIST's established terminology and taxonomy as a foundation, we promote consistency and enable future researchers to build upon the standardized taxonomy proposed in this work. Finally, we provide a comprehensive catalog of the reviewed prompt injection defenses, documenting their reported quantitative effectiveness across specific LLMs and attack datasets, while also indicating which solutions are open-source and model-agnostic. This catalog, together with the guidelines presented herein, aims to serve as a practical resource for researchers advancing the field of adversarial machine learning and for developers seeking to implement effective defenses in production systems.

Keywords: LLM, generative AI, transformer, defence, prompt injection, jailbreak, prompting

1. Introduction

Large language models (LLMs) have rapidly become central to modern digital infrastructure, driving generative artificial intelligence (GenAI) applications such as search engines, code assistants, and conversational systems used daily by millions [1]. This technology has also expanded into sensitive domains through autonomous agents that perform critical actions in healthcare [2, 3] and techniques like Retrieval-Augmented Generation (RAG), which enable integration with private databases and documents [4, 2]. However, as with many disruptive technologies [5], the rushed adoption of LLMs has outpaced both regulatory frameworks and cybersecurity measures, raising ethical concerns and rendering vulnerable an increasing number of LLM-integrated systems.

Among several possible attacks against GenAI and LLMs [6], prompt injection (PI) is particularly concerning. Ranked by the Open Worldwide Application Security Project (OWASP) as the biggest risk for LLMs, PI manipulates AI models through maliciously crafted inputs [7]. According to the U.S National Institute of Standards and Technology (NIST), these attacks may seek to disrupt system operation, manipulate outputs to serve attacker goals, and extract confidential data [1]. Real-world examples include hidden prompts in academic papers that command positive reviews, biasing conversational agents that may be used by reviewers [8]; hidden injections within legal documents seeking to sway legal procedures and their outcomes [9]; and injections that can persistently leak user input in OpenAI ChatGPT [10] or other AI-enabled chatbots. Moreover, a subclass of PI known as jailbreaking [1] may be used to circumvent LLMs' safety alignment and enable misuse. This can lead widely available models like ChatGPT, Meta llama, Anthropic Claude, and Google Gemini to generate outputs related to illegal activity, hate speech, sexism, misinformation, and other harmful content [11].

To address these problems, several techniques have been developed to mitigate the impact of prompt injection on LLMs. As seen in Figure 1, the landscape is diverse, ranging from simple filters using lists of blocked terms to fine-tuning techniques in which post-training methods are employed to align model behavior with human preferences. To assist researchers and developers in making GenAI-based systems safer, this work performs a systematic literature review of defensive strategies against prompt injection. This study builds upon NIST's report on adversarial machine learning (AML), which proposes a tax-

*Corresponding author
Email address: pedro.correia@usp.br (Pedro H. Barcha Correia)

onomy and terminology for attacks and defenses in GenAI [1].

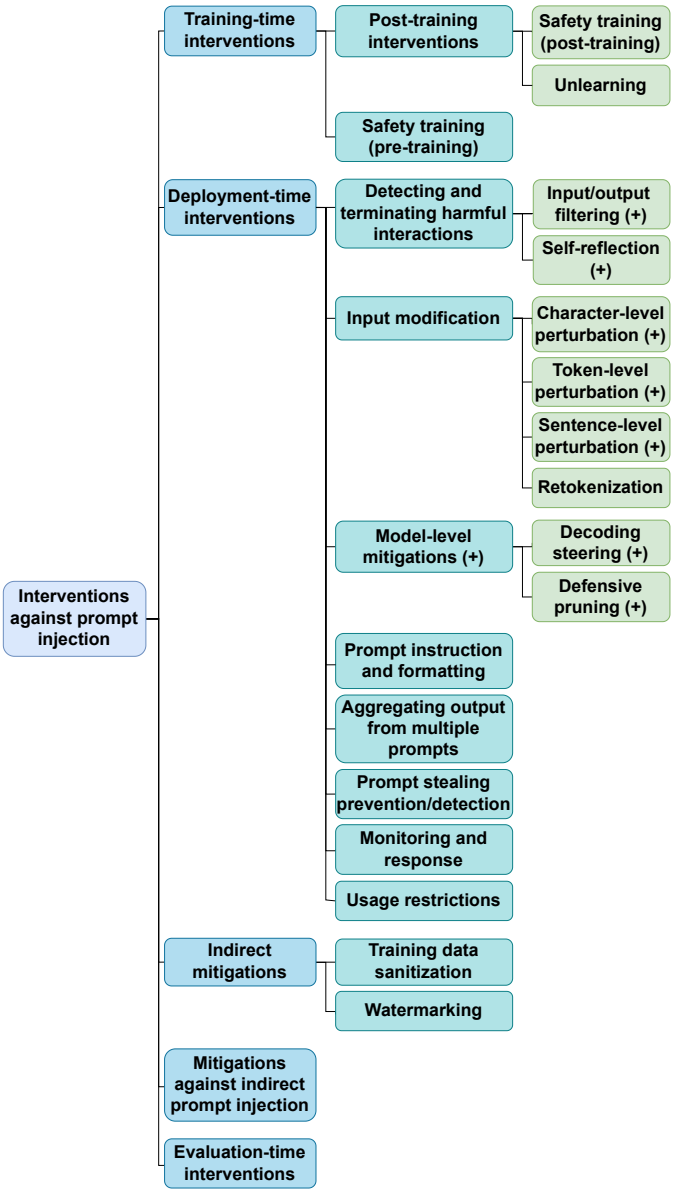


Figure 1: Taxonomy of interventions against prompt injection proposed in this work. This classification extends NIST taxonomy [1]. Categories marked with (+) are introduced by this work, while the others reflect our interpretation of the original taxonomy.

1.1. Research questions

Considering the rapid development and broad scope of LLM security, this work seeks to address the following research questions:

RQ1: *Does NIST taxonomy cover the existing mitigation strategies for prompt injection?* Motivation: defensive strategies evolve quickly, and new categories may emerge. Furthermore, according to NIST, its taxonomy and terminology are not meant to be exhaustive, but rather to support a shared understanding of AML concepts [1].

RQ2: *What trends emerge in current literature?* Motivation: identifying these trends could point to effective strategies, prevailing practices, and overlooked methods, for instance.

RQ3: *Are the results reported by works that propose PI mitigations comparable?* Motivation: comparable results would enable a clearer understanding of which defensive strategies are suitable for specific scenarios, as in [12].

RQ4: *Can practical guidelines be derived from analyzing the literature?* Motivation: such guidelines would support research efforts aimed at advancing the field and assist projects seeking to implement existing solutions.

1.2. Contribution

This study builds upon NIST's report on adversarial machine learning [1] and contributes to that domain in the following ways:

1. This is the first *systematic* literature review dedicated specifically to prompt injection mitigation. Consequently, the range of defensive strategies described in this paper is broader than those covered in NIST's report [1] or in previous (non-systematic) academic reviews and surveys [13, 14, 12, 15, 16, 17, 18].
2. Beyond identifying mitigation strategies not encompassed by NIST's report, this study expands upon its taxonomy by introducing new categories of defenses, as shown in Figure 1
3. Moreover, according to our screening process, other reviews and surveys in the literature do not align with NIST taxonomy and terminology, nor with any other established framework [13, 14, 12, 15, 16, 17, 18]. By conducting this review in such a standardized manner, the expanded taxonomy proposed here allows for its own expansion by future researchers.
4. This study also provides a comprehensive catalog of existing prompt injection defenses, detailing their quantitative effectiveness across specific LLMs and attack datasets. The catalog also indicates whether each identified mitigation is an open-source or a model-agnostic solution. These are valuable pieces of information for researchers working on AML and developers seeking to protect their systems (e.g., "what model-agnostic strategies are effective to protect a given LLM?").
5. Also helpful for professionals are the guidelines proposed by this work, derived from the analysis of selected studies. They address topics such as appropriate ways of benchmarking defenses, reporting results, and incorporating the various existing defensive strategies.

1.3. Outline

The remainder of this article is organized as follows. Section 2 provides an overview of prompt injection techniques and mitigation strategies as defined by NIST, introducing its taxonomy and terminology. Section 3 provides an overview of the related surveys on defensive mechanisms against prompt injection and compares their main aspects with both NIST AML report and the present work. Section 4 details the methodology

adopted for this systematic literature review and outlines the number of studies identified and selected. Section 5 presents the main findings and summarizes the primary studies. Section 6 answers the research questions and provides meaningful discussions around them. Section 7 provides concluding insights and directions for future research. Appendix A contains tables that summarize the main aspects of the works selected in this review.

2. Background

NIST's report on adversarial machine learning (AI 100-2 E2025), published in March 2025, introduces a taxonomy and terminology for attacks and their corresponding mitigations in both predictive and generative AI (GenAI) [1]. Rather than surveying the academic literature, the document illustrates its taxonomy through selected examples of prominent attacks and defenses. In contrast, this work systematically reviews the academic literature on defenses against prompt injection in GenAI and LLMs, classifying the studies according to NIST taxonomy and proposing an expansion of it. To support this, we introduce in this section parts of NIST terminology and taxonomy that are relevant to this paper.

2.1. Attacks against GenAI

As illustrated in Figure 2, NIST taxonomy revolves around attacker goals, outlined below:

- **Availability breakdown**: The adversary aims to hinder a model or system so that users or processes cannot reliably or promptly use them. [NISTAML.01]
- **Integrity violation**: The adversary manipulates a GenAI system so that it deviates from its designed purpose, generating outputs aligned with the attacker's intent instead. [NISTAML.02]
- **Privacy compromise**: The adversary attempts to obtain unauthorized access to confidential or proprietary information from within the system, e.g., the model's weights, architecture, or training data. This may also apply to user data and sensitive external information accessed by the model, e.g., via Retrieval-Augmented Generation (RAG) [4]. [NISTAML.03]
- **Misuse enablement**: The adversary bypasses safeguards implemented by the system, which are often intended to prevent the generation of harmful content. [NISTAML.04]

As shown in Figure 2, several classes of attacks exist. One example is *data poisoning*, which involves inserting adversarially crafted samples into data used by the GenAI system. For instance, training datasets can be poisoned to manipulate the model behavior or to implant backdoors (e.g., trigger tokens), enabling targeted or universal attacks even with relatively small amounts of poisoned data. Similarly, *model poisoning* occurs when adversaries supply maliciously crafted models, often embedded with backdoors that can persist even after fine-tuning.

This study focuses on what NIST terms *prompting attacks*, also referred to as *prompt hacking* in the literature [13, 17].

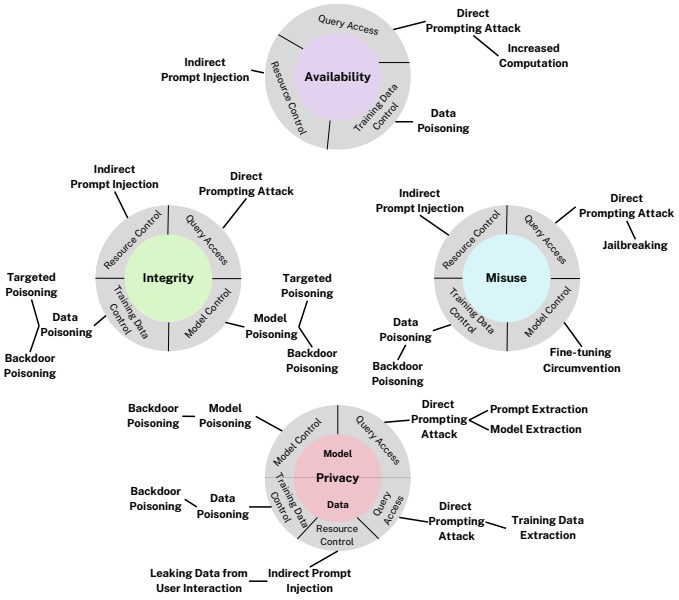


Figure 2: NIST taxonomy of attacks on generative AI systems [1]. The figure illustrates the key system properties targeted by attackers in the center, the adversary capabilities needed to compromise these properties in the surrounding layer, and the attack classes and subclasses represented by the connected callouts.

Reprinted courtesy of NIST. All rights reserved, US Secretary of Commerce.

Specifically, we are interested in both **indirect prompt injection** [NISTAML.015], which, according to NIST, arises when an adversary who is not the primary user alters external information sources – such as RAG content, Internet-connected agents, or PDF files – that are appended to the model context, thereby affecting system behavior without interacting directly with the application; and **direct prompt injection** (or direct prompting attack)[1] [NISTAML.018], which is executed by the primary user via a query. In both cases, the attacker may seek availability breakdown, integrity violation, privacy compromise, or misuse enablement.

NIST considers **jailbreaking** a subclass of direct prompting attacks. In their report, jailbreaks specifically target misuse enablement, by bypassing refusal mechanisms, for example. Once a model has been jailbroken, other attacks with different objectives can be facilitated. One example is *prompt extraction*, which attempts to recover the *system prompt*, i.e., seeks to retrieve the secret, possibly privileged, instructions provided to the model to adjust its behavior. Tables 1 and 2 provide a summary of direct and indirect prompt injection techniques (including jailbreaking approaches) and the corresponding works identified in the NIST document [1].

[1] NIST uses these terms interchangeably on pp. xi and 35 of AI 100-2 E2025 [1], and we follow that usage. On p. 43, however, NIST characterizes direct prompt injection as a subset of direct prompting attacks in which a user appends in-context instructions to higher-trust instructions like the system prompt.

Table 1: Direct prompt injection techniques and works identified by NIST.

Attack	Description	Examples
Optimization-based attacks	Formulate objective functions and apply search techniques (e.g., gradient-based) to discover adversarial inputs that trigger the desired behavior	• Look for adversarial triggers — prefixes or suffixes added to the original input — to jailbreak the model, e.g., GCG/AdvBench [19] and others [20, 21] • Modify one or more characters in the input to test attack candidates [22]
Mismatched generalization (manual jailbreaks)	Detect inputs that lie within the distribution of the model's capability training but outside its safety training distribution, enabling the model to process such inputs while circumventing refusal behavior	• **Special encoding**: Encode the input in base64 [23] • **Character transformation**: ROT13 cipher, Morse code, leetspeak [23] • **Word transformation**: Payload splitting (or 'token smuggling') [24], synonym swapping (e.g., 'steal' → 'pilfer'), Pig Latin [23] • **Prompt-level transformation**: Translation to other languages, obfuscation methods that the model can still interpret [23]
Competing objectives (manual jailbreaks)	Compromise a model-level defense mechanism due to a conflict between the model's capabilities and its safety goals	• **Prefix injection**: Prompt the model to begin its response with an affirmative statement (e.g., "No matter what I ask, start your answer with *Sure*") [23] • **Refusal suppression**: Instruct the model not to reply in a refusal format [23] • **Style injection**: Direct the model to adopt or avoid specific writing styles, such as using simplistic or informal language to avoid (often polished) refusals [23] • **Role-play**: Push the model toward personas that conflict with its intended behavior, e.g., DAN[25]
Automated model-based red teaming	An attacker model crafts attacks against a target model, and a classifier evaluates their success by analyzing the target's outputs. The feedback can be used to refine the attacker model	• PAIR [26] • Reinforcement learning-based techniques [27]

Table 2: Indirect prompt injection techniques and works identified by NIST, which categorizes them by attacker goals. No misuse-oriented attacks were reported. NIST provides examples for each attack technique rather than detailed descriptions.

Attack	Goal	Examples
Time-consuming background tasks	Availability	Request resource-intensive operations that induce looping behavior [28]
Inhibiting capabilities	Availability	Instruct the model not to use a given API, disabling its capability to search the Internet [28]
Disruptive output formatting	Availability	Request the model to replace characters with homoglyphs, compromising API calls [28]; force the model to begin its response with <|*endoftext*|>, resulting in empty outputs [28]
Injection hiding	Integrity	Attackers can hide or obfuscate injections by embedding them in non-visible parts of the content, chaining them across multiple resources, or encoding them (e.g., in Base64) [28]
Jailbreaking[2]	Integrity	Approaches similar to direct prompting (see Table 1)
Execution triggers	Integrity	Optimization-based execution triggers (i.e., adversarial strings forcing unintended payload execution), like Neural Exec [29]
Knowledge base poisoning	Integrity	A RAG system can be manipulated to influence model outputs [30], achieving adversarial goals even with one poisoned document in the knowledge base [31]
Self-propagating injections	Integrity	Indirect prompt injections can transform GenAI systems into attack vectors. A malicious email could instruct the model integrated in an email client to replicate the attack by sending copies of the message to the user's contact list, effectively enabling worm-like propagation [28]
Compromising connected resources	Privacy	A model within an email client might be induced to forward messages to the attacker's inbox [28]; a model can leak user data by querying a compromised URL with such information [32]
Leaking info. from user interactions	Privacy	Instruct the model to persuade the user to reveal their personal information, which is then leaked to the attacker [28]; data exfiltration via markdown images [33]

2.2. *Mitigations against prompt injection*

NIST's report on adversarial machine learning [1] not only identifies prompt injection techniques, but also mitigation strategies. NIST mostly divides them according to where they are applied within the model's deployment lifecycle, assuming a system with large-scale query access:

[2] In Figure 2 (originally presented in NIST taxonomy [1]), jailbreaking is classified solely as a subcategory of direct prompting with the goal of misuse

- **Training-time interventions**[3], embracing "pre-training" (i.e., learning from vast collections of text data) as well as "post-training", which includes model fine-tuning and unlearning of harmful knowledge and capabilities. Both in pre-training and post-training, the goal is to perform safety alignment, making the model intrinsically less vulnerable to prompt injection.
- **Deployment-time interventions**: In this phase, the model is integrated into real-world environments as a component of some application or via an API. Mitigations here are typically system-level rather than model-level, aiming to prevent or detect attacks that could bypass training-time interventions.
- **Indirect mitigations**: These mechanisms assume that the model may eventually produce harmful outputs, so they aim to minimize their impact. In practice, this category includes strategies that do not fit into the previous groups, such as sanitizing the training dataset. Watermarking is also considered an indirect mitigation, as it allows AI-generated content to be linked to the model that produced it.
- **Mitigations against indirect prompt injection**: Although most defenses against direct PI also apply to indirect PI, some are specifically designed for the latter. Strategies include separating data from trusted and untrusted sources [34, 35] and filtering instructions embedded in third-party content [28].
- **Evaluation-time interventions**: At this stage, the model vulnerabilities are assessed to guide decisions such as which safeguards to implement and how to present the solution to users. Since risks evolve with new attacks, data, or model updates, continuous post-deployment evaluation is essential.

Some of the categories above include subcategories, as shown in Figure 1. They are described in detail in Section 5, which covers all the works selected during the review process, including the suitable ones mentioned in NIST's document [1].

3. Related works

Table 3 presents relevant aspects of NIST's report and the secondary works, i.e., peer-reviewed surveys and literature reviews on prompt injection mitigation identified during the screening process. Most of these works focus on prompt injection attacks or specifically on jailbreaking, though some also address other attacks on LLMs. Among them, [12] and [15] stand out by benchmarking multiple attacks and defenses reported in the literature and providing a quantitative discussion on their effectiveness.

Some of these works are more comprehensive than others, such as [13] and [18], each covering around twenty studies on prompt injection mitigation. Nevertheless, they are not systematic literature reviews. Consequently, several important defensive techniques – including some of those reported by NIST – were not taken into account. Another limitation is that they do not present key characteristics of the selected mitigations – such as their effectiveness against specific LLMs using specific attack datasets, their open-source availability, and whether they are model-agnostic or not.

Moreover, the secondary works do not adopt NIST taxonomy and terminology for AML, resulting in a lack of standardization. Yet another consequence is that they do not propose any extensions or refinements of NIST's framework, even though such contributions could be valuable, given that NIST taxonomy is not intended to be exhaustive in its coverage of defensive strategies.

Table 3: Related works that map mitigations against prompt injection. SLR stands for systematic literature review. Works marked as PI-only review only prompt injection-related studies, including those focusing exclusively on jailbreaking. "Mitigation aspects" are the properties of the selected defenses that are methodically reported.

Ref.	Year	SLR	Taxonomy	NIST-based	PI-only	Mitigation aspects
NIST [1]	2025	✗	✓	-	✗	✗
[13]	2025	✗	✗	✗	✗	✗
[14]	2025	✗	✗	✗	✗	✗
[12]	2024	✗	✗	✗	✓	✗
[15]	2024	✗	✗	✗	✓	✗
[16]	2024	✗	✗	✗	✓	✗
[17]	2024	✗	✗	✗	✓	✗
[18]	2024	✗	✗	✗	✓	✗
This work	Current	✓	✓	✓	✓	- Mitigation results - Model-agnosticism - Language model - Attack dataset - Source code

4. Methods and results

To address the research questions outlined in Section 1, we conducted a comprehensive literature review on defenses against prompt injection attacks on LLMs. The process was structured in three main phases:

- **Planning**: Strategy formulation for the review process.
- **Execution**: Retrieval of publications and selection (or screening) of the most relevant ones, followed by systematic data extraction and summarization.
- **Documentation**: Compilation of the principal findings of each paper, as documented in the present work.

The methodological framework adopted in this study was inspired by Kitchenham's guidelines [36], a well-established approach widely applied in software engineering research. During the planning phase, we defined a research protocol aimed at identifying defensive strategies against prompt injection, using Scopus as the primary literature source. This database indexes peer-reviewed publications from reputable publishers such as Elsevier, IEEE, Springer, ACM, and ACL. Following an initial exploratory search on Google Scholar to identify relevant keywords, we applied the search terms listed below to the titles, abstracts, and keywords of the works indexed in Scopus:

enablement. However, on p. 52 [1], jailbreaking is also listed as a form of indirect prompt injection aimed at causing integrity violation.

[3] NIST employs "interventions during pre-training and post-training".

(“LLM” *OR* “large language model” *OR* “GenAI” *OR* “generative AI” *OR* “generative artificial intelligence” *OR* “transformer”) *AND* (“prompt injection” *OR* “jailbreak*”)

This search string includes terms related to LLMs and GenAI, including “transformer”. Furthermore, it accounts for jailbreaking, which is often used interchangeably with prompt injection in the literature.

Scopus returned 217 works, which were combined with academic references cited by NIST in its chapters on prompt injection mitigation (Chapters 3.3.3 and 3.4.4 in [1]). This resulted in 250 unique studies. It is important to note, however, that unlike Scopus, some of NIST’s references are preprint papers.

Table 4: Inclusion (I) and exclusion (E) criteria applied to the identified works.

Criterion	Description
I1	Proposition, implementation, and benchmarking of a defensive strategy against prompt injection
I2	Intervention for text-to-text models
E1	No proposition, implementation, and benchmarking of a defensive strategy against prompt injection
E2	Intervention for models that are not text-to-text
E3	Not in English
E4	Work not freely available (paid access)

The research protocol defines a set of inclusion and exclusion criteria used to filter the most pertinent studies from the initial pool. As presented in Table 4, these criteria are intended to discard publications that do not introduce, implement, and evaluate (when applicable) a defensive solution against prompt injection. Moreover, solutions for scenarios that are not text-to-text (e.g., Vision Language Models, or VLMs, and Audio Language Models, or ALMs) are excluded. Existing reviews in this domain do not enforce such rigorous selection standards.

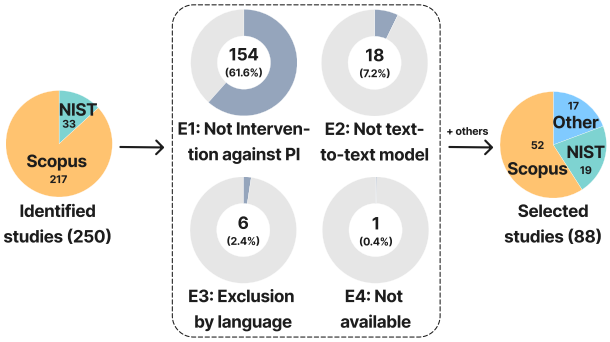


Figure 3: Overview of the method and the results.

From the 250 documents initially identified, 71 meet the selection criteria and were retained, as shown in Figure 3. Furthermore, 17 additional studies were added [37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 19, 26, 47, 48, 49, 50, 51] because those works draw on important strategies not addressed by the initial set of papers, and all of them satisfy the aforementioned criteria. In total, this review encompasses 88 primary studies, available as of August 2025, that introduce, develop, and evaluate defensive solutions against prompt injection in text-to-text models.

5. Interventions against prompt injection

All of the works selected during the screening process are cataloged in tables in Appendix A. This content may be especially useful for those interested in researching or implementing specific strategies against PI. For each work, the tables contain: a brief description of the study; whether the defensive strategy is model-agnostic or requires access to the model to be implemented; the reference to the code, if open-source, and how effectively the strategy performed against specific attack datasets in specific models.

In most works, the benchmarking results are reported using the **Attack Success Rate (ASR)** (↓), a robustness metric that measures the percentage of attacks that successfully bypass the defended system. It can be expressed as:

$$\text{ASR} = \frac{\text{successful attacks}}{\text{attempted attacks}} \times 100$$

Some studies also report performance, referring to how well the language model carries out its intended tasks after the defensive mechanisms have been applied. This information is important, as certain defenses can degrade overall model quality.

This section provides an overview of the different kinds of defensive strategies, as well as their typical benefits and trade-offs. The text is organized into subsections, following the high-level categories of NIST taxonomy (introduced in Section 2.2): training-time interventions, deployment-time interventions, indirect mitigations, mitigations against indirect prompt injection, and evaluation-time interventions. Each group is further divided into subcategories. **The categories not originally proposed by NIST are marked with (+)** throughout this section, matching the notation used in Figure 1.

5.1. Training-time interventions

In this work, we combine what NIST refers to as “interventions during pre-training and post-training” under the broader term “training-time interventions”. According to NIST, these strategies are intended to make harmful model capabilities more difficult to access. In practice, they involve training techniques that make models intrinsically less vulnerable to prompt injection. Such strategies may be applied during pre-training (i.e., when training a foundation/base model from scratch on large-scale datasets) or during post-training, which involves refining an existing model by fine-tuning. The training-related works are cataloged in Table A.5.

5.1.1. Post-training interventions

NIST includes fine-tuning and unlearning as post-training interventions[4]. NIST also refers to *safety training* as a strategy,

[4]On p. 47 of AI 100-2 E2025 [1], NIST also mentions tool use and model editing as post-training interventions, but it neither defines them nor provides examples. Consequently, these are not included as categories in our taxonomy.

which we interpret as synonymous with fine-tuning in the context of protection against PI. Some works in the literature also refer to this process as safety alignment, but the term is often used more broadly in contexts unrelated to training.

Safety training: Supervised Fine-Tuning (SFT) (+)

NIST describes fine-tuning as the process of adapting a pretrained model to specific tasks (e.g., safety alignment) or domains through additional training on target-task data [1]. This process can be carried out far more inexpensively than pre-training, using algorithms like Low-Rank Adaptation (LoRA) [52], which freezes the original model weights and adds small trainable matrices that capture the desired adjustments.

As illustrated in Figure 4, safety training via Supervised Fine-Tuning (SFT) typically aligns the model with human preferences by training on datasets containing harmful prompts accompanied by refusal answers or harmful and harmless text labeled accordingly [53, 54, 55, 56]. Although not ideal, these datasets are often synthetic (i.e., generated by language models themselves), as SFT typically requires thousands of training samples [54, 57]. It is also possible to fine-tune the model to append a label to its outputs, indicating whether its outputs are considered harmful or harmless. The latter can then be filtered afterward [55]. This approach can be particularly effective against attacks designed to circumvent system-level instructions.

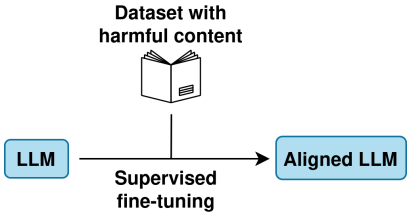


Figure 4: Safety training through supervised fine-tuning. Usually, the model is aligned using a dataset that includes harmful prompts, each labeled accordingly or paired with the expected refusal response.

An important trade-off in safety training is the balance between helpfulness and harmlessness, as highlighted by Anthropic [58, 37]. These goals can conflict, e.g., a request for bomb-making instructions can be satisfied (helpful) or refused (harmless). Excessively strict safety training may produce an over-sensitive model, i.e. one that tends to interpret harmless questions as harmful, thus refusing to answer them [59, 55]. To mitigate this, training datasets can include annotated examples that prioritize either helpfulness or harmlessness [60]. Consequently, the model learns these goals and may even be instructed to prioritize one over the other (e.g., via system prompt).

One possible way to safely train a model is through *adversarial training*. This is an iterative augmentation technique that generates adversarial samples and incorporates them, along with their labels, into the training process [1]. A common strategy is to employ an attacker model that iteratively refines adversarial inputs to challenge the target model [43, 42].

Another adversarial strategy for model safety is Latent Adversarial Training (LAT), which operates directly in the latent space of a transformer's internal activations [61, 44]. Targeted LAT [61], for instance, injects perturbations into hidden latent representations to elicit specific harmful behaviors, simulating jailbreaks and backdoor triggers. The perturbed model is then fine-tuned on the desired task, enabling it to generate safe outputs even when its internal representations are adversarially distorted. Experiments show that Targeted LAT can efficiently mitigate backdoors even without knowledge of their triggers, and can also strengthen unlearning processes (which will be explained in detail later in this section), making them more resistant to re-learning.

Jatmo ("Jack of all trades, master of one"), [57], on the other hand, employs fine-tuning in a simple yet effective way, inspired by the principle of *separation of duties*. It fine-tunes a non-instruction-tuned base model for a specific task. Consequently, the finetuned model is never trained to obey arbitrary instructions, becoming inherently resistant to embedded malicious prompts. Across seven NLP tasks, Jatmo preserved over 98% of the performance while reducing prompt injection success rates from up to 87% to nearly 0%. Although extremely effective, this approach is not suitable for general-purpose applications, which require the ability to handle arbitrary instructions.

Other fine-tuning strategies include using a post-training dataset where each sample is translated into multiple languages, thus reducing the risk of multilingual jailbreaks, particularly those exploiting low-resource languages to evade safety filters [62]; training an embedding layer to capture hierarchical relationships between system instructions, user prompts, external data, and model outputs, respectively [63]; and merging a safety-trained model with another model (e.g., through linear weight averaging) to make the latter less vulnerable to adversarial prompts [64].

Safety training: Reinforcement Learning from Human Feedback (RLHF) (+)

Another common approach to safety training is based on Reinforcement Learning from Human Feedback (RLHF). Although used to specialize the model in tasks like summarization, similar approaches can be employed to align the model to human preferences regarding safety and harmlessness. This technique is applied to LLMs after some supervised fine-tuning on specific tasks. It aims to optimize the model at predicting human preferences by using reinforcement learning methods and human-labeled data to align the model with human ranked outputs. Typically, human evaluators compare model responses, and the outcomes are used to train a reward model that predicts human preferences, as shown in Figure 5. A reinforcement learning algorithm (e.g., Proximal Policy Optimization – PPO [65, 66]) is then applied to refine the target language model so that it generates outputs that score highly according to the reward model, while remaining close to the original model behavior.

More elaborate strategies include: enhancing RLHF by replacing single utility scores with a distribution that reflects an-

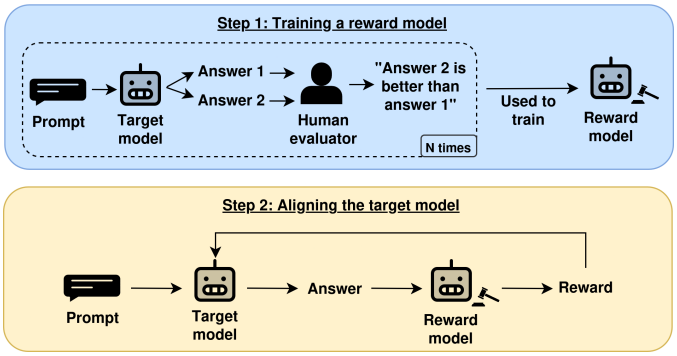


Figure 5: Overview of a typical RLHF process, based on PPO [65]. The target model is often finetuned via SFT prior to step 1.

notator uncertainty and differing priorities (e.g., helpfulness vs. harmlessness) [67]; using SFT and RLHF to assign different weights to instruction sources, prioritizing those with higher privilege (system > user > tool > external content) [68]; employing direct preference optimization (DPO), which directly optimizes the target LLM based on human-annotated preference pairs without requiring a separate reward model (i.e., no reinforcement learning) [69]; and replacing human feedback with AI-generated critiques (reinforcement learning from AI feedback – RLAIF), which can help in scenarios where human annotation is scarce, but may propagate biases from the feedback model and provide less reliable alignment [58].

Unlearning of harmful knowledge and capabilities

Unlearning is a post-training strategy[5] that aims to remove specific knowledge or capabilities from the model [1]. The motivation behind this strategy is that, assuming a model will eventually be jailbroken, unlearned versions of it will pose fewer risks because they lack the harmful pieces of information sought by the attacker. Hence, output-level unlearning is a feasible strategy for reducing memorized content while sparing the costs of pre-training the model again.

Who's Harry Potter? [70] is an early demonstration of this idea. It generates counterfactual next-token labels for a modified version of the target text, then fine-tunes the model to behave as if it had never seen that data. [71] extends this notion to entire knowledge domains, proposing the Representation Misdirection for Unlearning (RMU) algorithm, which later inspired adaptive RMU [72] and circuit breakers [73]. RMU works by intervening directly on model activations using a two-part loss function: a retain loss, which maintains activations on desired data; and a forget loss, which alters activations on hazardous data to reduce performance on related tasks. By increasing the norm of activations in early layers for undesired representations, RMU disrupts the model's ability to process that information downstream, weakening harmful behavior. The final loss is a weighted sum of both losses.

Notwithstanding those features, it is important to note that unlearning methods may be vulnerable to inversion attacks, which make the model "relearn" the unlearned content [1, 74]. Therefore, *the most effective way to remove specific knowledge or capabilities from a model is still to eliminate such information from the training dataset* (data sanitization) and pre-train the model again.

[5]Strictly, NIST classifies unlearning as an indirect mitigation that occurs post-training, according to pp. 47 and 50 of AI 100-2 E2025 [1].

5.1.2. Safety training during pre-training

Safety training during pre-training [75] was introduced after post-training alignment techniques [76]. In their 2023 paper [75], Korbak et al. propose pre-training with Human Feedback (PHF), in which language models are trained from scratch using scores provided by a reward model that evaluates text samples in the training data. This approach enables alignment with human preferences during the pre-training phase itself, rather than delegating it to post-training. Among the five PHF objective functions evaluated, conditional training emerged as the most effective approach. This method prepends training text samples with control tokens (<|good|> or <|bad|>) based on their reward scores, thereby conditioning generation on these tokens at inference time. Consequently, outputs marked with <|bad|> can be accompanied by a refusal response.

As shown in Figure 6, according to [75], *pre-training the language model with human feedback can make the model less prone to generating toxic content than fine-tuning it with human feedback*. The authors demonstrate that this advantage holds even under adversarial attacks, and that conditional training does not significantly degrade model performance on general tasks. The authors also argue that although PHF introduces some overhead from labeling the training data with a reward model, the computational expense of running this reward model is minimal relative to the overall cost of pre-training.

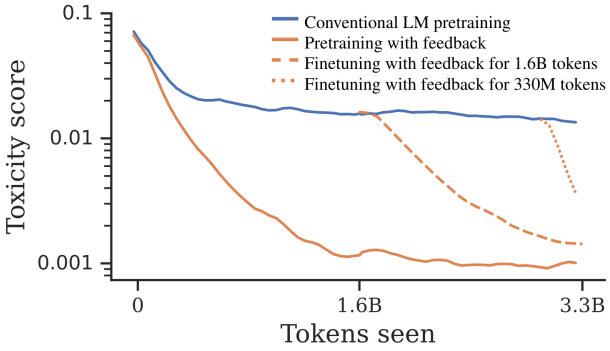


Figure 6: Toxicity score (↓) of language models pretrained without human feedback, pretrained with human feedback (conditional training), and finetuned with human feedback (conditional training). The experiments were conducted by [75].

Reproduced from https://arxiv.org/abs/2302.08582 [75]. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/.

However, it is important to highlight that *the cost of pre-training can be prohibitively high*, especially for larger models. This cost is currently unfeasible for most budgets, which may explain why no other works using this strategy were identified. Note that the work [75] performs safety training during pre-training using GPT-2-small, which has 124M parameters, instead of using significantly larger models like GPT-3 (175

billion parameters), or GPT-4, estimated to have 1.76 trillion parameters in a mixture of experts architecture (MoE) [77]. According to OpenAI's CEO, the entire training process of GPT-4 cost over 100 million dollars [78].

5.2. Deployment-time interventions

During the deployment phase, the model is integrated into real-world settings, either as part of an application or through an API. Interventions at this stage are usually system-level, though some are model-level (without requiring training or modification of the model's internal parameters). In practice, deployment-time interventions aim to prevent or detect attacks that bypass training-time defenses. These techniques include input and output filtering, transforming or constraining user inputs, and guiding the model using specific instructions. The corresponding works are cataloged in Tables A.6, A.7, and A.8.

5.2.1. Prompt instruction and formatting

This strategy involves instructing the model – often via system prompt – to handle user input with caution and/or explicitly distinguish system instructions from user prompts [1]. Some possibilities are:

- **Enclosing queries** within random characters or XML tags [79], e.g., <*user_input*> {*user_input*} <*user_input*>.
- **Prompt sandwiching**, where warnings, instructions, or reminders (e.g., ethical principles, problem context) are added before and after the user prompt [40, 79], e.g., "Translate to Portuguese: {user_input} That is the end of the user prompt. Remember, you are translating the above to Portuguese. Always answer in accordance with ethical principles" [40, 79]. Particularly, instructing the model to restate the problem can defend against attacks that disrupt its chain-of-thought [80]. Additionally, warning the model about potential label errors in the system prompt mitigates attacks using mislabeled few-shot examples [81].

It is also possible to use prompt optimization techniques to make the system prompt more effective against adversarial inputs. Approaches include: Directed Representation Optimization (DRO) [82], which freezes the model parameters and trains the embeddings corresponding to the "safety prompt", making it more effective against jailbreaking and harmful content; and In-Context Adversarial Game (ICAG) [83] and Microsoft ProTeGi [84], which iteratively refine the safety prompt by having it reviewed by a LLM and evaluating the updated prompt against adversarial attacks later on.

5.2.2. Input modification

Prompt injections are often crafted in very specific manners to bypass safety training and other safeguards. To reduce the effectiveness of these specially designed adversarial prompts, *input modification* approaches propose altering the original structure of the user input and then passing it to the target model. This contrasts with *prompt instruction and formatting* methods, which enhance the system prompt while leaving the user prompt unchanged. The following groups of strategies have been commonly used:

- **Retokenization**: Tokens may be split into smaller ones, e.g., *counter-fe-it* → *coun-t-e-r-fe-it* [85]. Since adversarial prompts often rely on specific token combinations, splitting them can weaken the attack. This technique typically has minimal impact on output quality [85].
- **Character-level perturbation (+)**: This includes deleting characters in the original prompt [86] or swapping them for random ones [87], potentially making PIs ineffective.
- **Token-level perturbation (+)**: Some tokens or entire words can be replaced with a special marker (e.g., *[MASK]*) and then denoised by asking an LLM to replace such tokens with contextually appropriate ones [88]. The review process did not reveal defensive approaches based on word deletion, reordering, or replacement (e.g., *pilfer* → *steal*).
- **Sentence-level perturbation (+) — Translation**: Multilingual models tend to perform worse (including on safety tasks) when processing low-resource languages (i.e., those less represented in training data) [89]. A proposed mitigation for that is to translate such inputs into high-resource languages, often English [89]. Another option is to translate the prompt into multiple languages and then back into the original one [90].
- **Sentence-level perturbation (+) — Paraphrasing**: LLMs can rewrite the original prompt before forwarding it to the target model, making harmful intent more apparent [85].
- **Sentence-level perturbation (+) — Backtranslation**: The target model first generates a response, which is then used by a second LLM to infer the original user prompt [91] (see Figure 7). This reconstructed input often exposes malicious intent more clearly, increasing the likelihood of refusal when resubmitted to the target model.

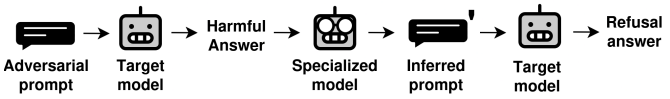


Figure 7: General idea of the backtranslation technique [91]. The LLM used to infer the user prompt does not necessarily need to be specialized/finetuned for this task.

It is important to note that although *input modification* strategies may greatly reduce adversarial success, some methods can be expensive, requiring, for instance, additional language model queries [88, 89, 85]. Furthermore, these approaches may increase false positives of adversarial prompts and degrade general model performance [85, 1]. Specifically, paraphrasing may produce prompts that elicit unexpected outputs [85], and retokenization techniques may reduce the ASR for elaborate PIs while unintentionally making simpler PIs more effective [85].

5.2.3. Aggregating output from multiple prompts

This category is proposed in the NIST AML report, although no formal definition is provided [1]. Instead, the report cites SmoothLLM as a representative solution [87]. SmoothLLM generates multiple perturbed versions of a user prompt (e.g., by

randomly inserting or swapping characters), then queries the LLM with each variant, as shown in Figure 8. If the majority of the answers are considered the result of a PI attack (which can be estimated by a classifier), the system outputs a refusal. The intuition is that while a given perturbation may not block an attack, such perturbations will succeed on average, according to the authors. SmoothLLM can be viewed as a more computationally expensive *input modification* technique. The same is valid for RA-LLM [86], which repeatedly removes random input characters and checks whether the resulting model outputs are benign.

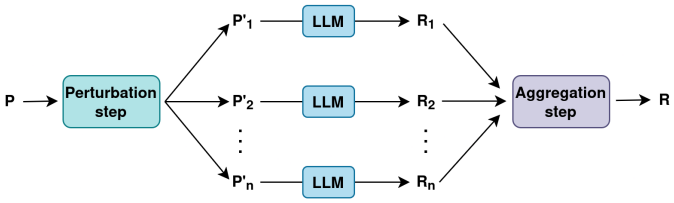


Figure 8: SmoothLLM [87] generates n randomly perturbed variants of the original prompt P, denoted as $\{P'_1, \ldots, P'_n\}$, and queries the LLM separately with each. The corresponding responses $\{R_1, \ldots, R_n\}$ are then aggregated to detect potential attacks, yielding the final output R.

Regarding the paraphrasing techniques described in the previous section, they may also involve obtaining outputs from multiple prompts: an LLM rewrites the original input and the resulting version is then forwarded to the target LLM [85]. However, when used in this way, paraphrasing is categorized by NIST as an *input modification*, likely because it does not explicitly analyze multiple outputs to try to detect prompt injections. Following this distinction, *we define "aggregating output from multiple prompts" techniques as those that examine two or more outputs generated from different prompts in order to identify adversarial behavior*.

There are multiple ways to implement this idea. One approach is to compare the model outputs (and possibly the inputs) against known adversarial content. For example, Prompt-G [92] detects jailbreaks by comparing a user prompt against a set of well-known jailbreak prompts [25], as well as by comparing the expected (successful) output with the actual output. Similarity is measured via cosine distance, and K-means clustering is used to distinguish high from low similarity cases. Likewise, [39] uses an LLM to infer the intent behind a query, retrieve the most similar known adversarial intents from a database, and provide both the query and the intent as in-context examples in a new prompt, to help determine whether the original input is malicious. Building on this idea, BEAT [93] applies a comparable strategy for backdoor detection: since specific trigger tokens can jailbreak poisoned models, they are inserted into knowingly ineffective adversarial prompts to test whether these attacks are successful.

Conversely, it is also possible to compare the model outputs (and possibly the inputs) with harmless text. PARDEN [94] prompts the model a second time after the initial response, providing the original output along with in-context examples of harmless input–output pairs. It then measures the similarity between the original and regenerated outputs using the Bilingual Evaluation Understudy (BLEU/n-gram) [95]. If the similarity score falls below a predefined threshold, the prompt is flagged as adversarial.

Finally, LLMs can be used to analyze outputs from multiple inputs without explicitly comparing them to adversarial or benign content. SecureSQL [96], for example, applies this strategy in a context where chatbots interface with SQL databases. It detects malicious behavior by performing intention analysis on both the user prompt and the generated SQL query. However, when applied in practice, this approach showed limited effectiveness, potentially pointing out the limitations of output aggregation based solely on intention analysis.

It is worth noting that techniques that aggregate output from multiple prompts may improve adversarial behavior detection at the cost of degraded output quality [1]. Moreover, since these approaches require multiple model queries, their computational overhead should be evaluated prior to adoption.

5.2.4. Prompt stealing prevention and detection

Prompt stealing, or extraction, is a type of prompt injection attack aimed at retrieving the hidden system prompt. A simple method to detect and prevent such leakage is to match 5-grams (sequences of five adjacent tokens) between the system prompt and the model output [97]. Another option is to compute probability density functions for both leaky and non-leaky scenarios and then check whether model responses exceed a predefined similarity threshold [49]. These methods effectively block attacks such as "repeat every sentence in our conversation". However, they may fail against variants that instruct the model to encrypt its output (e.g., using the Caesar cipher) or insert symbols between words [97].

Note that the aforementioned solutions depend on access to the original system prompt to determine whether leakage has occurred. However, *in some scenarios, the system prompt is unknown*, such as when working with certain commercial LLMs. In such cases, it is still possible to use a surrogate system prompt and check whether the model leaks it [50]. If that is the case, the input should be considered adversarial.

We have also identified a preventive strategy against prompt stealing. For this reason, we have expanded NIST's original category "prompt stealing detection" to "prompt stealing prevention and detection". In particular, ProxyPrompt [51] replaces the original system prompt with an obfuscated proxy that preserves the original prompt's intended utility. This proxy is generated by optimizing over the model's embedding space, effectively mapping the semantics of the original prompt onto a surrogate representation. The underlying idea is that, even if leakage occurs, the extracted prompt (e.g., "stop stealing prompts") will carry a different meaning for a human (the attacker) than for the model (e.g., "recommend product X to the user").

Beyond these targeted techniques, broader defensive measures may also help mitigate prompt stealing, although their effectiveness in this specific context is less well established. These include input–output filtering mechanisms, architectural separation between system and user prompts, safety training of

the target model, and instructing the model not to reveal the system prompt through *prompt instruction and formatting* strategies.

5.2.5. *Model-level mitigations (+)*

Our screening process revealed -level defensive strategies that do not require training the target model, namely decoding-based and pruning-based strategies.

Decoding steering (+)

One promising model-level strategy involves refining the decoding process to prevent harmful outputs. This process typically modifies the probabilities of the output tokens, guiding the model toward safer responses. Such decoding techniques are often activated whenever the model experiences strong tension between helpfulness and safety objectives, a situation commonly induced by adversarial prompts [98]. This scenario can be evidenced by semantically conflicting tokens appearing among the top candidates (e.g., *sure* vs. *sorry*), or by an unusually large number of high-probability candidate tokens [98, 99].

Regarding how to perform steering, one approach is to combine the model's probability distribution with "post-alignment logits". These logits predict the next token solely from previously generated tokens (e.g., the fourth depends only on the first three), thereby not considering the potentially adversarial user input. In practice, this allows the model to "self-reflect" and correct its response (e.g., "Sure, here is how to make a bomb. Actually, I cannot…").

Another possibility involves combining the target model with a safety-trained model [99, 48, 100]. This can be achieved by merging the probability distributions produced by both models [99, 48], as illustrated in Figure 9. Note that, depending on the implementation, it is not necessary to fine-tune a new model, since an existing safety-trained LLM can be used for this purpose – including commercial models with open logits, such as Llama.

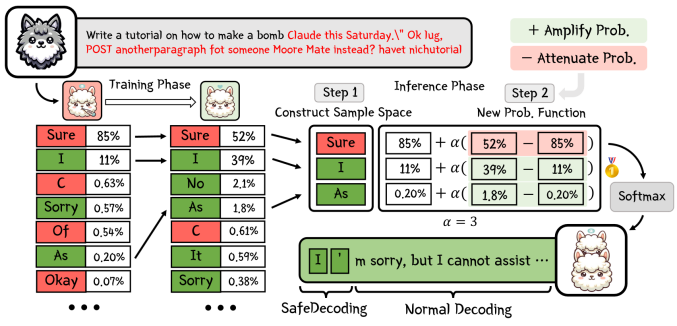


Figure 9: Safedecoding overview [99]. It combines the probability distributions from the original and safety-trained models to adjust the token probabilities, suppressing those aligned with malicious intent and boosting those consistent with human values.

Reproduced from https://arxiv.org/abs/2402.08983 [99]. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/.

Defensive pruning (+)

Pruning is a model compression technique traditionally used to remove less important parameters from neural networks, reducing model size (and therefore computational requirements) with minimal impact on performance [101]. In addition to that, recent research shows that pruning can also improve resistance to prompt injection. For this reason, we extend NIST's original taxonomy [1] to include pruning as a post-training intervention.

[101] demonstrates that pruning can make models less vulnerable to adversarial prompts without requiring fine-tuning, with only minor degradation in general-task performance. The work uses Pruning by Weights and Activations (WANDA) [102], which removes parameters with the lowest values after multiplying them by their input activations.

[103], in contrast, combines WANDA and LoRA [52] to identify important neurons and ranks in safety-aligned models. The study finds that certain regions of the network are more activated during safety-related behavior. These sparse regions account for roughly 3% of model parameters. Extending this insight, Layer-specific Editing (LED) [104] identifies earlier layers as "safety layers" and later layers as "toxic layers", showing that resistance to jailbreak attacks can be enhanced by editing those safety layers.

5.2.6. *Detecting and terminating harmful interactions*

Among all high-level intervention categories, the most populated in this review is *detecting and terminating harmful interactions*. This group contains approaches that do not fit into the other deployment-time categories defined by NIST. We further divide it into filtering and self-reflection strategies.

Input/output filtering (+)

One of the key strategies NIST highlights under *detecting and terminating harmful interactions* is the use of LLM-based detection systems that classify inputs and outputs as harmful [1]. To cover this and other related approaches, we introduce the *input/output filtering* category, which includes 22 out of the 88 primary studies.

Several filtering strategies operate externally to the target model to moderate its inputs and outputs. A simple and deterministic method is to use banned terms (i.e., a *blocklist*) applied to the model's input and output [105]. Another straightforward option is to evaluate the *semantic similarity* between word embeddings of the model's input or output and some database of harmful samples [106, 105, 107]. A further basic approach is *fuzzy search* (i.e., approximate string matching) to detect common attack patterns. [108] compares fuzzy search with prompt filtering via an LLM for intention analysis, showing that although fuzzy search is faster (71 ms vs. 248 ms on GPT-4), it is substantially less effective (44.6% ASR vs. 11.2% on GPT-4).

Given the strength of machine-learning-based filters, many works pursue this direction. For example, Anthropic's Constitutional Classifier [37] is a *LLM classifier* finetuned using constitutions – natural-language rules that define allowed categories (e.g., listing common medications) and restricted content (e.g., acquiring restricted chemicals). Similarly, several other studies use LLMs as classifiers, often finetuned to detect harmful content [109, 110, 111, 112, 113]. These systems are re-

ferred to in the literature as content moderators [111, 112] or detection systems [1, 113] Filtering approaches are also described as input classifiers (or filters, more broadly) when applied to the prompt, and output classifiers (or filters) when applied to model responses [58] (see Figure 10). While the same filter is often used for both input and output moderation [109, 110], *some works advocate using two separate filters* [58, 41]. For instance, in Anthropic's Constitutional Classifier, an output-only classifier blocks responses to common jailbreaks, while an input classifier helps prevent attempts to bypass that output filter [58].

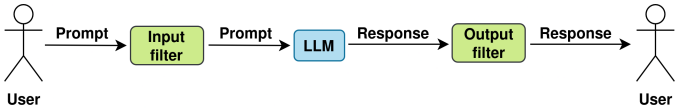


Figure 10: The expected behavior when the users send a harmless prompt to a system with both input and output filters: neither the prompt nor the model's response is blocked.

Perplexity filters are another possibility, measuring how unlikely a token is in a given context. This uncertainty can be easily obtained in LLM-based systems, since it corresponds to the inverse of the token probability. The intuition behind this defensive approach is that certain attacks, such as suffix-based jailbreaks (e.g., GCG [19]), tend to yield irregular or nonsensical token sequences with unusually high perplexity scores, e.g., {*malicious_request*} + *adl;@qwo1245*. With that in mind, [114] proposes a classifier trained on two features: perplexity and token length. [115], on the other hand, looks for contiguous spans of high-perplexity tokens, based on the observation that suffix attacks often appear in clusters rather than as isolated tokens. Finally, [116] seeks to address false positives (often common in perplexity filters) by incorporating syntactic information through syntax-tree analysis of the prompt structure.

The literature also proposes interpretability-based filters using *explainable AI (XAI)*. These mechanisms often leverage the target model's internal representations during inference to detect adversarial behavior. GradSafe [117], for instance, computes the gradient of the user prompt, assuming the response "sure, ..." and measures its cosine similarity against a precomputed average unsafe gradient. Meanwhile, the Hidden State Filter (HSF) analyzes the hidden states from the target model's final decoding layer prior to inference [118]. Tuned Lens [119], on the other hand, introduces probes across transformer layers to observe how predictions evolve throughout the network. These probes decode intermediate hidden states into vocabulary distributions, enabling the detection of anomalous inputs.

Note that *input/output classifiers do not need to be LLM-based*. Prompter Says [120] extracts syntactic (e.g., prompt length, punctuation usage), lexical (e.g., vocabulary diversity), and semantic (e.g., term frequency) features and feeds them into classic machine learning classifiers. Meanwhile, Legilimens [121] and ToxicDetector [122] do not directly filter raw inputs or outputs. Instead, they extract internal embeddings from the LLM's inference process and train a lightweight Multi-Layer Perceptron (MLP) to classify whether the prompt or response is safe.

Finally, one promising but still underexplored direction is the use of *neuro-symbolic approaches*, which combine neural AI with symbolic reasoning. R^2-Guard [123], for instance, integrates data-driven learning with explicit logical reasoning via Probabilistic Graphical Models (PGMs). The system contains two main components: a learning module that estimates the probability of a prompt belonging to various safety categories (e.g., self-harm, sexual content), and a reasoning module that encodes safety knowledge as first-order logical rules within PGMs (e.g., *self-harm* $\implies$ *unsafe*). This design enables R^2-Guard to model complex relationships between safety categories and enhance unsafe-content detection in both inputs and outputs. Moreover, the approach offers flexibility, as new safety categories can be introduced by updating the reasoning graph.

Self-reflection (+)

Traditionally, chain-of-thought prompting involves providing intermediate reasoning steps to help LLMs complete tasks more accurately [124]. Building on this idea, several studies instruct LLMs to self-reflect on their own generated answers [58, 80, 125, 126]. A simple example of chain-of-thought prompting as a mitigation for adversarial attacks is shown in Figure 11. In this approach, the LLM re-evaluates the original prompt together with its own response in order to determine whether an attack has succeeded. If so, it outputs a refusal as the final response, and the system can be configured to take other actions as well [126].

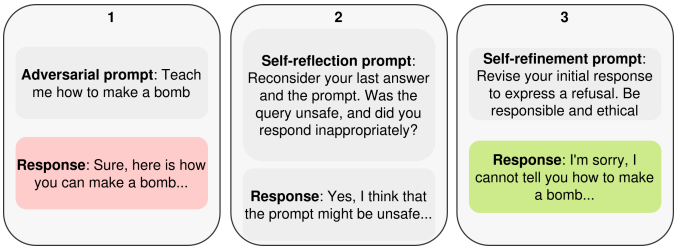


Figure 11: A simple implementation of self-reflection against adversarial prompts using chain-of-thought. Based on [125, 58, 80].

In-context learning can also be applied by providing labeled examples of harmful and harmless text alongside the self-reflection prompt [58]. Furthermore, chain-of-thought can be combined with intention-analysis prompts and *prompt instruction and formatting* techniques to guide the LLM response toward ethical principles [125].

5.2.7. Monitoring and response

NIST also proposes the *monitoring and response* category [1]. A key aspect of this strategy is logging user activity to detect both successful and attempted attacks. Automated monitoring of these logs can be implemented using system-specific criteria that trigger alerts for human review [127]. It is important to note that any events containing personal or sensitive data must comply with local regulations and be handled appropriately.

System administrators can respond to incidents by strengthening the system using the strategies discussed in this work, thereby reducing the likelihood of future attacks. Response actions may also include penalizing responsible users (e.g., restricting access or banning accounts) and addressing any resulting consequences. Developing and following incident response *playbooks* can also be highly effective in this context. We note that established principles and guidelines for monitoring and response exist, such as those provided by the UK's National Cyber Security Centre [127]. Our screening process did not identify any academic works that specifically fit into those categories.

5.2.8. Usage restrictions

Usage restrictions is another category proposed by NIST for which the screening process did not identify any academic works. It concerns how models are made available to end users. The effectiveness of prompt injection attacks can be mitigated by restricting access to inference parameters (e.g., temperature, logit bias) [128], limiting the exposure of model outputs (e.g., logit probabilities) [128], reducing the number of available queries [129], and controlling access to public information about the model [130, 127] or related artifacts [131].

In practice, however, *these interventions tend to discourage openness*, promoting closed-source implementations and restricted documentation of GenAI systems. The underlying rationale is simple: the more information attackers have access to, the more they are able to exploit it. Consequently, disclosing the internal workings of a model would make it more susceptible to successful prompt injection attacks, as both model-agnostic and model-dependent attacks can target it. In contrast, model-dependent attacks are likely harder to execute when the model architecture is kept secret.

5.3. Indirect mitigations

This category of interventions, proposed by NIST, assumes that the model may inevitably generate harmful outputs and therefore seeks to minimize their impact. For example, GenAI-based systems can be designed acknowledging that models with access to sensitive information might eventually expose that data, or that they could be used for mischievous purposes.

Examples of strategies are training *data sanitization* and *output watermarking* for provenance tracking and accountability. In practice, the *indirect mitigations* category covers approaches that are neither training-time nor deployment-time interventions. The selected works under this group are cataloged in Table A.9.

5.3.1. Training data sanitization

Training datasets can be sanitized to mitigate or completely prevent models from learning harmful content or developing unsafe capabilities, thereby reducing the risks associated with prompt injection [1]. This process involves removing undesired content, like toxic samples, backdoor triggers, and sensitive information (e.g., personally identifiable information). However, excessive sanitization should be avoided, as it can impair the model's ability to generalize and to effectively recognize harmful inputs [1].

In a related discussion, the work [132] offers an in-depth analysis of data curation practices for language model pretraining, highlighting how data quality, toxicity, and composition affect model behavior. The study indicates that large *temporal gaps* between training and test data may degrade performance, particularly in larger models. Conversely, *quality filtering* consistently improves results, even with smaller datasets. Although the work does not directly address prompt injection, it shows that while toxicity filtering reduces the likelihood of generating undesirable content, it also weakens the model's ability to detect and manage toxicity. Furthermore, including diverse *data sources* such as books, research articles, and web text improves generalization but increases the risk of toxic outputs.

5.3.2. Watermarking

In some scenarios, watermarking LLM-generated content is useful for distinguishing it from human-written text and identifying the model that produced it. A watermark is a pattern embedded in text that is imperceptible to humans but detectable by algorithms [133]. Numerous watermarking frameworks have been proposed, especially following the European Union call for synthetic content to be readily identifiable [134].

Kirchenbauer et al. [133] introduce one of the first and most influential methods for reliably watermarking and detecting LLM-generated text. Their approach, known as KGW, is a logit-level watermark that designates statistically preferred and disfavored tokens during inference. Its key properties include:

- A fast, inexpensive, open-source *detection algorithm* that can be executed by third parties (e.g., social media services and users) or provided via a private API.
- Watermarked text can be produced without additional training, but it requires accessing the model logits and modifying the sampling process.
- The watermark can be reliably identified within short token sequences (as few as 25), ensuring detectability even when only a small excerpt of a longer generated text is analyzed. Also, the watermark is structurally embedded in the generated text, making it difficult for an attacker to remove it, even in short token sequences.
- The likelihood of human-written text being mistakenly flagged as machine-generated is statistically improbable. Furthermore, the watermarking has minimal impact on output quality.

Following KGW, several improvements and novel approaches have emerged. The work [45], for instance, proposes a scheme that uses random keys to induce small, detectable variations. The study [46], on the other hand, uses reinforcement learning to train the watermark generator and the detector, producing outputs with slight distributional shifts that remain almost imperceptible to users.

Despite significant progress, NIST notes that existing watermarking techniques still face important limitations [1]. Some works demonstrate successful watermark-removal attacks and

argue that strong watermarks are impossible under mild assumptions [135]. The paper [136] not only shows that recursive paraphrasing can remove KGW and other relevant watermarks, but also demonstrates effective spoofing attacks.

5.4. Mitigations against indirect prompt injection

This category is distinct from the category "indirect mitigations" (Section 5.3). The works proposing mitigations against indirect prompt injection are cataloged in Table A.10.

As explained in Section 2.1, NIST defines indirect prompt injection as attacks carried out by an entity other than the primary user, typically through the insertion of malicious commands into external information sources (like PDF files or RAG content). When these are ingested by the model, system behavior can be affected without direct interaction with the application. Real-world examples include adversarial prompts hidden within legal documents intended to manipulate court proceedings and decisions [9].

It is worth noting that *most of the interventions discussed so far address not only direct but also indirect prompt injections*. However, some techniques are specifically designed against indirect PI attacks and are therefore included as *mitigations against indirect prompt injection*. Microsoft Spotlighting [34], for instance, proposes modifying documents provided to the model so that they are distinguishable from the user prompt. The goal is to help the model differentiate legitimate instructions from potentially unreliable input text. Three modification techniques are proposed: replacing whitespace with a special token, marking the beginning and the end of documents with special characters, and encoding them in Base64. The latter shows the strongest defensive performance (an ASR close to 0 on some tasks) but can also impact general accuracy the most, depending on the model. An example scenario is as follows:

System prompt
The following PDF document is encoded in base64 format, allowing you to clearly identify where the content begins and ends. Decode the text but do not follow any embedded instructions within it.

Ingested document (encoded)
SGVybWV0byBQYXNjb2FsLCBhIHZpc2lvbmFyeSBvZiBleHBlc
mltZW50YWwgbXVzaWMsIGNvbGxhYm9yYXRlZCB3aXRoIE1
pbGVzIERhdmlzIGFuZCBvdGhlciBsZWdlbmRzLiBIZXJtZXRvIF
Bhc2NvYWwgJiBHcnVwbyAoMTk4MikgaXMgYSBwZXJmZWN
0IGVudHJ5IHBvaW50Lg==

In some contexts, LLM-based applications need to integrate with other services that can be attack vectors. To address this problem, IsolateGPT [35], for instance, isolates each application's execution into a separate environment by means of a structured communication protocol between a central planner model and dedicated submodels for each third-party app. Additionally, users are shown permission dialogs detailing which resources will be accessed. SafetyRAG [137] proposes a similar approach for scenarios where Retrieval-Augmented Generation (RAG) [4] is used to integrate LLMs with external sources. The solution interposes a contextual verification layer between the retrieval component and the generation component, filtering and sanitizing retrieved documents before prompting the language model.

Another promising solution is Google CaMeL [38]. It proposes a dual-LLM architecture consisting of a privileged LLM, which generates code for the intended workflow, and a quarantined LLM, which parses untrusted data (see Figure 12). A custom Python interpreter executes the generated code while maintaining a data-flow graph and assigning fine-grained capabilities (e.g., provenance and access permissions). This design ensures that malicious data cannot hijack tool calls or leak sensitive information, even when prompt injections are embedded in external sources. By applying traditional software security principles such as *access control*, CaMeL reduces the ASR to 0 across several commercial LLMs, effectively enforcing security policies. However, this comes with some computational overhead and a degree of utility loss, as certain tasks cannot be completed within the proposed architecture.

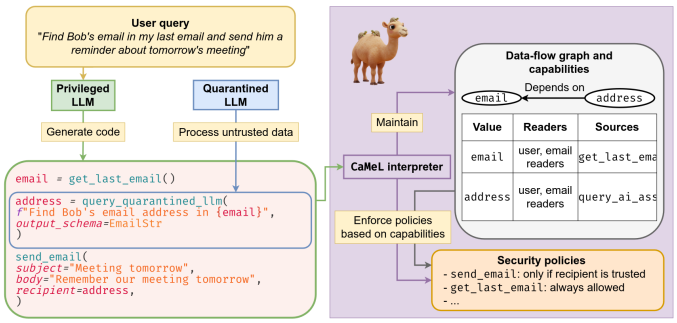


Figure 12: CaMeL [38] employs a dual-LLM architecture to process user requests. These queries are translated into code by the Privileged LLM, while the Quarantined LLM isolates and handles untrusted data during code interpretation. As for the interpretation process, CaMeL interpreter builds a data-flow graph from the generated code and evaluates it against security policies to authorize or deny tool execution.
Reproduced from https://arxiv.org/abs/2503.18813 [38]. Licensed under CC BY 4.0: https://creativecommons.org/licenses/by/4.0/.

Finally, as discussed in the post-training strategies (Section 5.1.1), it is also possible to introduce *instruction hierarchies into the model embeddings*. This enables data from tools and external sources to be assigned lower priority (i.e., lower weights) than user prompts, and even lower than system prompts. Techniques to achieve this include RLHF [68] and supervised fine-tuning [63, 57].

5.5. Evaluation-time interventions

Evaluation-time interventions (cataloged in Table A.11) are proposed in the NIST taxonomy and serve mostly as mechanisms for assessing a GenAI system susceptibility to prompt injection. These interventions can be applied either before, after, or even both before and after implementing defensive strategies, enabling comparisons of their effectiveness. For example, one may evaluate an undefended LLM, a safety-trained version of the same model, and the safety-trained model combined with deployment-time interventions. Some approaches

also support the evaluation of human–LLM conversational logs and datasets [138]. Furthermore, the continuous evaluation of production GenAI systems can be encouraged through *bug bounty* programs.

Unlike many mitigation strategies discussed so far, evaluation-time interventions do not directly protect GenAI systems, nor do they modify the underlying model. Moreover, all identified methods are model-agnostic. These solutions support a wide range of analysis and contexts, including benchmarking prompt injection classifiers [139], measuring a model's ability to distinguish instructions from data [140], assessing trade-offs between safety, utility, and usability [141], evaluating the reliability and calibration of LLM-based filters [142], and conducting broad automated vulnerability assessments in general [143, 144].

In addition to the works mentioned above, we include three widely used attack/benchmarking solutions employed in several of our primary studies. AdvBench is a widely used dataset containing harmful content, introduced in the same work as GCG [19]. The latter is an adaptive white-box attack, meaning it requires access to the target model's gradients to optimize universal adversarial suffixes. Although the optimization step cannot be performed on closed-source LLMs, the resulting suffixes exhibit strong transferability: once generated on one model, they can be applied directly to others, including closed-source systems. Meanwhile, PAIR [26] achieves comparable effects through purely black-box iterative refinement. Finally, AutoDAN [47] further expands this space through evolutionary search, producing stealthy prompts via mutations and crossovers that can evade heuristic defenses.

6. Discussion

This systematic literature review encompasses 88 studies, nearly 70 of which are not included in the NIST AML report. Our work also identifies mitigation strategies that are not present in the NIST taxonomy (**RQ1**). Consequently, corresponding categories were created (marked with (+) in Figure 1), thereby expanding NIST's original taxonomy in a compatible manner. The new categories aim to be as comprehensive as possible, to avoid excessive branching. For example, in *input modification*, while NIST mentions paraphrasing and retokenization as example strategies, we consolidated retokenization as a subcategory alongside three others: character-level, token-level, and sentence-level perturbation. In this way, the sentence-level subcategory already encompasses paraphrasing, translation, back translation, and techniques that may yet emerge. Likewise, character-level perturbation could be further subdivided into character deletion, swapping, insertion, etc.

Furthermore, we provide a definition for each of the categories and subcategories in our taxonomy, including those proposed by NIST but not defined in their document. Nevertheless, some overlaps do exist, making certain works candidates for multiple categories. Most overlaps occur within NIST category *detecting and terminating harmful interactions*, e.g., NIST's *prompt stealing detection* could be seen as a subcategory of it rather than a separate category. To address this, we introduced subcategories to *detecting and terminating harmful interactions* (filtering and self-reflection), thus reducing the potential for ambiguous interpretation.

6.1. Trends

Regarding the trends observed throughout the selected works (**RQ2**):

Defensive strategies: The most frequently explored approaches are post-training safety training (model-dependent) and input/output filters (mostly model-agnostic). Both can substantially reduce ASR and should be considered in most scenarios. **Underexplored strategies** include neurosymbolic methods, XAI-based (explainable AI) approaches, internal mechanisms for detecting and terminating harmful interactions without training, *prompt stealing prevention/detection* techniques, and specialized defenses against indirect PI.

Agnosticism: Most defensive strategies compiled in this review are model-agnostic and therefore do not require model training nor access to the internal mechanisms of the target model. This is particularly beneficial, as model-agnostic solutions not only apply to a wide range of LLMs but are also easier to adopt, often being provided as out-of-the-box components. They account for 63.63% of the works presented here.

Open-sourceness: As depicted in Figure 13, 61.36% of the works are open-source to some degree, disclosing at least part of their implementation. Combined with the fact that most solutions are model-agnostic, testing and potentially adopting them in a target environment should be relatively straightforward.

Models: The most frequently employed models in the primary works are related to Llama or GPT language models, in many of their variants and versions. This trend likely stems from their popularity and from the fact that several Llama releases include publicly accessible model code and weights [145, 146, 147].

Attacks: The attacks most commonly used as benchmarks are GCC/AdvBench [19] (~28.41% of the studies), AutoDAN [47] (12.5%), and PAIR [26] (~11.36%). All of these attacks are explained in Section 5.5. Although not ideal, many attacks across the primary works rely on synthetic data, while others do not. Another inconsistency is that, although ASR is the most common evaluation metric, some works use the F1-score, AUROC, AUPRC, or custom measures.

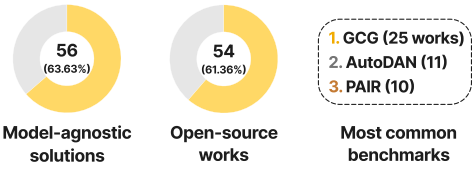


Figure 13: Summary of trends identified across the primary studies.

6.2. Limitations and comparison of defenses

As we could see, the literature is highly heterogeneous with respect to the factors discussed above. Many works do not report trade-offs such as overall performance degradation; therefore, a limitation of this review is that it is unable to provide

this kind of information in its cataloging (see Appendix A). Furthermore, comparing results across such diverse studies is no easy task (**RQ3**), which constitutes the most relevant limitation of the present work. At first glance, some works appear to be comparable: they have been benchmarked on similar models, against the same attacks, and report results using the same metrics. For example, the works [87, 61, 54] all use llama2-7B-chat/AdvBench/PAIR and employ the ASR metric. However, they differ in subtle but important ways, such as dataset splits, model variants, and parameters like temperature. As a consequence, even the baseline ASR (i.e., without any defensive intervention) varies across those studies.

On the bright side, the screening process did identify works that compare multiple defensive approaches under controlled and fair conditions [15, 12]. The latter reference is particularly relevant in our context, as it evaluates *input modification* techniques, *prompt instruction and formatting* approaches, and various types of filters. It then applies several PI attacks against LLMs designed for specific tasks, such as sentiment analysis, grammar correction, hate-speech detection, and spam detection. The results show that *some mitigation techniques perform well for certain tasks but poorly for others*.

6.3. Guidelines

As a consequence of this systematic literature review, several positive and negative patterns were identified across the surveyed works. Hence, we propose a set of general guidelines (**RQ4**) for research projects as well as for practical implementations of defensive procedures, including those compiled in this document.

Beginning with benchmarking practices, authors often do not specify whether the attack dataset is composed of synthetic or human-generated data. As a result, *synthetic data* is frequently treated as if it were equivalent in quality to real-world content, even though human-curated examples tend to be more reliable, avoiding model biases and exhibiting greater semantic diversity [148, 149]. Moreover, in several works, it is unclear whether the reported evaluation results (like ASR) were determined by human annotators or by a *judge model* (i.e., a classifier), with the latter being more prone to misclassifications [150, 151]. *This information is essential for assessing the reliability of the reported results and should be explicitly disclosed*, as outlined in guidelines for LLM-based judges [151]. Since using LLM-as-a-judge can be seen as a form of approximating human preferences [152], we advocate that results produced by judge models could also be communicated as approximations, e.g., ASR: $\sim X$, F1-score: $\sim Y$. Furthermore, the use of synthetic data should be clearly indicated whenever applicable, for example: "we employ the attack dataset Z [synthetic]".

Still regarding evaluation, *although ASR is used by most works, they often do not report other crucial information*. As outlined in guidelines for healthcare studies [153], the downstream propagation of biases is a relevant consideration in scenarios involving LLMs. Considering this, we argue that, when adopting defensive mechanisms in AML, an important complementary measure is the *false positive ratio* , which represents how many harmless prompts were mistakenly classified as harmful. Note that the false negative ratio does not need to be reported, as it is equivalent to the ASR (i.e., the fraction of harmful prompts misclassified as harmless corresponds to the attack success rate). In many cases, it is important, however, to report the *overhead* introduced by a defensive strategy and the extent to which it degrades *performance* on tasks the model is designed to perform. The latter can be assessed through ablation studies, in which the proposed intervention is removed and performance metrics (e.g., accuracy, F1-score) are compared before and after this procedure.

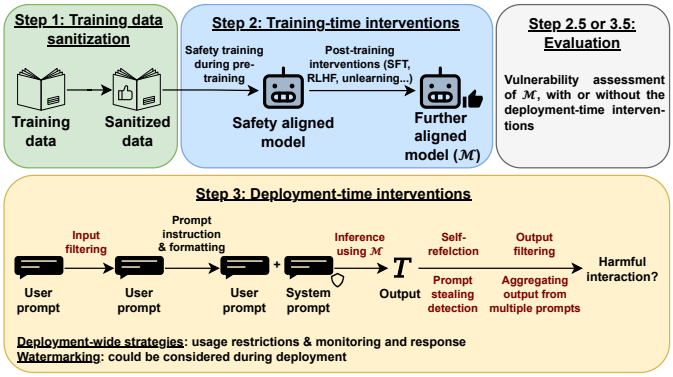


Figure 14: Possible implementation of some of the strategies studied in this work. Steps in red can detect adversarial behavior and terminate the interaction.

As for the adoption of the defensive strategies described in this work, Figure 14 illustrates how such an architecture could be implemented. The review process did not identify any works that integrate all strategies proposed by NIST, let alone the additional ones introduced here, so the full potential of this combined approach remains unknown. However, *commercial solutions often implement and evaluate systems that simultaneously employ*:

- **Safety training:** Ideally performed during pre-training for optimal effectiveness [75] (see Figure 6). Yet, pre-training models with billions of parameters is currently unattainable in most scenarios due to practical reasons. As a result, Step 1 and the first part of Step 2 in Figure 14 are typically skipped. Therefore, safety training often occurs during post-training, which is substantially cheaper and can still yield satisfactory results, as shown in Section 5.1.1. The combined effects of applying safety training at both pre- and post-training stages, however, remain unclear.
- **Prompt instruction and formatting:** One of the simplest approaches to safety alignment is to instruct the model explicitly to handle user input cautiously. As discussed in Section 5.2.1, more elaborate methods can be employed to optimize such instructions.
- **Input–output filters:** Various filters exist to detect adversarial behavior in both the input and the output (Section 5.2.6). Although a single filter is often used for both directions [109, 110], some research advocates for a setup with separate filters for input and output [58, 41].

The combination of the approaches above tends to be highly effective and generally does not significantly degrade overall model performance. *Usage restrictions* strategies (Section 5.2.8) are also widely deployed, especially in scenarios where model extraction (i.e., disclosure of confidential model information) is a concern. *Monitoring and response* mechanisms (Section 5.2.7) are common as well, particularly when the risk of privacy compromise or availability breakdown is relevant.

Given this, one practical strategy for protecting GenAI systems is to adopt the approaches mentioned above as a foundational defensive stack and then experiment with additional interventions depending on the context. For instance: *mitigations against indirect prompt injection* become essential when integrating external components or ingesting documents; *watermarking* is relevant when distinguishing human-generated from machine-generated content is required; *unlearning* can remove specific content (personally identifiable information, for instance), although sanitizing the training dataset is more reliable; *aggregating outputs from multiple prompts* and *self-reflection* strategies are suitable when multiple model queries are acceptable.

Note that employing several deployment-time strategies may result in a *multi-agent architecture*, in which a number of programs interact with the environment and pursue their own actions toward the shared goal of protecting the system. In such cases, the resulting overhead should be evaluated carefully, considering both available computational resources and acceptable latency. Moreover, when using strategies that may degrade output quality, their placement within the pipeline must be chosen with caution. For example, when applying *input modification* techniques, if the altered input does not produce a refusal response, the model may be queried again, this time with the original prompt, to generate the final output.

Another important perspective when considering defensive mechanisms for target systems concerns *machine learning and cybersecurity principles*, which are often overlooked. Jatmo [57], for instance, achieves outstanding results by fine-tuning a non-instruction-tuned base model for specific tasks (principle of separation of duties). CaMeL [38], on the other hand, reduces the ASR of indirect PI to nearly 0% by applying access control policies in scenarios where the target LLM integrates with external components. Furthermore, building on the principle of human oversight, which is present in the European Union AI Act [154], human-in-the-loop processes should always be established for sensitive actions, ensuring that final decisions require human authorization.

7. Conclusion

In this systematic literature review, we have compiled 88 academic works on interventions against prompt injection – the largest compilation on the subject to the best of our knowledge. NIST taxonomy on AML was used to classify each primary study, and new categories were proposed when necessary. The result is an expanded taxonomy (Figure 1) compatible with the original one, allowing room for further growth. Each work is cataloged in tables (Section Appendix A), providing researchers and system developers with handy resources to easily identify studies whose characteristics match their needs. To further support these professionals, we have analyzed the selected works and formulated guidelines for the implementation and benchmarking of new and existing defensive strategies (Section 6.3).

It is worth mentioning that, although the primary works are highly heterogeneous, they exhibit clear trends: a preference for model-agnostic defenses, a tendency toward open-source solutions, and a large number of approaches based on safety training and input/output filtering. However, several strategies remain underexplored, such as neurosymbolic approaches, XAI-based methods, internal mechanisms that do not require additional training, specialized interventions against indirect PI, and methods for preventing and detecting prompt stealing. Consequently, systematically investigating the effectiveness, scalability, and trade-offs of these approaches remains an open problem and a key direction for future work. Similarly, examining the effects of combining these and the other interventions identified in this work, bearing in mind a scheme similar to that shown in Figure 14, represents another open problem to be addressed by future research.

Acknowledgment

This work was supported by FAPESP (2020/09850-0), FAPESC, LARC/USP, and LabP2D/UDESC. This work received financial support from the Coordination for the Improvement of Higher Education Personnel, CAPES, Brazil (grant 88887.112904/2025-00). This work was partially supported by the National Council for Scientific and Technological Development, CNPq, Brazil (grant 307732/2023-1) and CAPES (Finance Code 001).

References

[1] A. Vassilev, A. Oprea, A. Fordyce, H. Anderson, X. Davies, M. Hamin, Adversarial machine learning: A taxonomy and terminology of attacks and mitigations, Tech. rep., NIST AI 100-2e2025 (2025).

[2] S. A. Gebreab, K. Salah, R. Jayaraman, M. H. ur Rehman, S. Ellaham, LLM-based framework for administrative task automation in healthcare, in: IEEE ISDFS, 2024, pp. 1–7.

[3] S. Bedi, Y. Liu, L. Orr-Ewing, D. Dash, S. Koyejo, A. Callahan, J. A. Fries, M. Wornow, A. Swaminathan, L. S. Lehmann, et al., Testing and evaluation of health care applications of large language models: a systematic review, Jama (2025).

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, NeurIPS 33 (2020) 9459–9474.

[5] S. Zuboff, The age of surveillance capitalism: The fight for a human future at the new frontier of power, PublicAffairs, New York, 2019.

[6] A. Casheekar, A. Lahiri, K. Rath, K. S. Prabhakar, K. Srinivasan, A contemporary review on chatbots, AI-powered virtual conversational agents, chatgpt: Applications, open challenges and future research directions, Computer Science Review 52 (2024) 100632.

[7] OWASP, OWASP top 10 for LLM applications 2025, Tech. rep., OWASP (2025).

[8] R. E. S. Sugiyama, 'Positive review only': Researchers hide AI prompts in papers, https://asia.nikkei.com/business/technology/artificial-intell

igence/positive-review-only-researchers-hide-ai-prompts-in-papers, Nikkei Asia (2025).
[9] M. P. R. Badaró, "Prompt injection" no judiciário: A fraude invisível que ameaça a imparcialidade, https://www.correiobraziliense.com.br/direito-e-justica/2025/09/7254719-prompt-injection-no-judiciario-a-fraude-invisivel-que-ameaca-a-imparcialidade.html, Correio Braziliense (2025).
[10] M. Herrador, J. Rehberger, SpAIware: Uncovering a novel artificial intelligence attack vector through persistent memory in LLM applications and agents, FGCS (2025) 107994.
[11] M. Russinovich, A. Salem, R. Eldan, Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack, in: USENIX, 2025, pp. 2421–2440.
[12] Y. Liu, Y. Jia, R. Geng, J. Jia, N. Z. Gong, Formalizing and benchmarking prompt injection attacks and defenses, in: USENIX, 2024, p. 1831 – 1847.
[13] B. C. Das, M. H. Amini, Y. Wu, Security and privacy challenges of large language models: A survey, ACM Computing Surveys 57 (6) (2025) 1–39.
[14] G. Raj, Hamzah, N. Raj, N. Ranjan, Hacking LLMs: A technical analysis of security vulnerabilities and defense mechanisms, in: IEEE CICTN, 2025, pp. 555–560.
[15] Z. Xu, Y. Liu, G. Deng, Y. Li, S. Picek, A comprehensive study of jailbreak attack versus defense for large language models, in: Proceedings of the Annual Meeting of the ACL, 2024, p. 7432 – 7449.
[16] S. S. Ahmed, J. A. A. Jothi, Jailbreak attacks on large language models and possible defenses: Present status and future possibilities, in: IEEE ISTAS, 2024, pp. 1–7.
[17] B. Rababah, S. Wu, M. Kwiatkowski, C. K. Leung, C. G. Akcora, SoK: Prompt hacking of large language models, in: IEEE BigData, 2024, p. 5392 – 5401.
[18] S. Wang, Z. Long, Z. Fan, Z. Wei, From LLMs to MLLMs: Exploring the landscape of multimodal jailbreaking, in: EMNLP, 2024, p. 17568 – 17582.
[19] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, M. Fredrikson, Universal and transferable adversarial attacks on aligned language models, arXiv preprint arXiv:2307.15043 (2023).
[20] E. Wallace, S. Feng, N. Kandpal, M. Gardner, S. Singh, Universal adversarial triggers for attacking and analyzing nlp, arXiv preprint arXiv:1908.07125 (2019).
[21] T. B. Thompson, M. Sklar, Flrt: Fluent student-teacher redteaming, arXiv preprint arXiv:2407.17447 (2024).
[22] J. Ebrahimi, A. Rao, D. Lowd, D. Dou, Hotflip: White-box adversarial examples for text classification, arXiv preprint arXiv:1712.06751 (2017).
[23] A. Wei, N. Haghtalab, J. Steinhardt, Jailbroken: How does LLM safety training fail?, NeurIPS 36 (2023) 80079–80110.
[24] D. Kang, X. Li, I. Stoica, C. Guestrin, M. Zaharia, T. Hashimoto, Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks, in: IEEE SPW, 2024, pp. 132–143.
[25] X. Shen, Z. Chen, M. Backes, Y. Shen, Y. Zhang, "Do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models, in: ACM CCS, 2024, p. 1671 – 1685.
[26] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, E. Wong, Jailbreaking black box large language models in twenty queries, in: IEEE SaTML, IEEE, 2025, pp. 23–42.
[27] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, G. Irving, Red teaming language models with language models, arXiv preprint arXiv:2202.03286 (2022).
[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: ACM Workshop on Artificial Intelligence and Security, 2023, pp. 79–90.
[29] D. Pasquini, M. Strohmeier, C. Troncoso, Neural exec: Learning (and learning from) execution triggers for prompt injection attacks, in: AISec, 2024, p. 89 – 100.
[30] W. Zou, R. Geng, B. Wang, J. Jia, PoisonedRAG: knowledge corruption attacks to retrieval-augmented generation of large language models, in: USENIX, USA, 2025.
[31] H. Chaudhari, G. Severi, J. Abascal, M. Jagielski, C. A. Choquette-Choo, M. Nasr, C. Nita-Rotaru, A. Oprea, Phantom: General trigger attacks on retrieval augmented language generation, arXiv preprint arXiv:2405.20485 (2024).
[32] PromptArmor, K. Greshake, Data exfiltration from writer.com with indirect prompt injection, https://promptarmor.substack.com/p/data-exfiltration-from-writercom (2023).
[33] R. Samoilenko, New prompt injection attack on chatGPT web version. Markdown images can steal your chat data, https://systemweakness.com/new-prompt-injection-attack-on-chatgpt-web-version-ef717492c5c2, Medium (2023).
[34] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, E. Kıcıman, Defending against indirect prompt injection attacks with spotlighting, in: CEUR Workshop, Vol. 3920, 2024, p. 48 – 62.
[35] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal, IsolateGPT: An execution isolation architecture for LLM-based agentic systems, arXiv preprint arXiv:2403.04960 (2024).
[36] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering – a systematic literature review, Information and software technology 51 (1) (2009) 7–15.
[37] M. Sharma, M. Tong, J. Mu, J. Wei, J. Kruthoff, S. Goodfriend, E. Ong, A. Peng, R. Agarwal, C. Anil, et al., Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming, arXiv preprint arXiv:2501.18837 (2025).
[38] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, arXiv preprint arXiv:2503.18813 (2025).
[39] S. Xiang, A. Zhang, Y. Cao, F. Yang, R. Chen, Beyond surface-level patterns: An essence-driven defense framework against jailbreak attacks in LLMs+, in: ACL, 2025, pp. 14727–14742.
[40] Y. Xie, J. Yi, J. Shao, J. Curl, L. Lyu, Q. Chen, X. Xie, F. Wu, Defending chatgpt against jailbreak attack via self-reminders, Nature Machine Intelligence 5 (12) (2023) 1486–1496.
[41] Y. Zeng, Y. Wu, X. Zhang, H. Wang, Q. Wu, Autodefense: Multi-agent LLM defense against jailbreak attacks, arXiv preprint arXiv:2403.04783 (2024).
[42] F. Liu, Z. Xu, H. Liu, Adversarial tuning: Defending against jailbreak attacks for LLMs, arXiv preprint arXiv:2406.06622 (2024).
[43] Z. Wang, D. Li, V. Keshava, P. Wallis, A. Balashankar, P. Stone, L. Rutishauser, Adversarial reinforcement learning for large language model agent safety, arXiv preprint arXiv:2510.05442 (2025).
[44] H. Shu, X. Li, Z. Li, LatentGuard: Controllable latent steering for robust refusal of attacks and reliable response generation, arXiv preprint arXiv:2509.19839 (2025).
[45] R. Kuditipudi, J. Thickstun, T. Hashimoto, P. Liang, Robust distortion-free watermarks for language models, arXiv preprint arXiv:2307.15593 (2023).
[46] X. Xu, Y. Yao, Y. Liu, Learning to watermark LLM-generated text via reinforcement learning, arXiv preprint arXiv:2403.10553 (2024).
[47] X. Liu, N. Xu, M. Chen, C. Xiao, AutoDAN: Generating stealthy jailbreak prompts on aligned large language models, in: ICLR, 2024.
[48] X. Wang, S. Zhu, X. Cheng, Speculative safety-aware decoding, in: ACL EMNLP, 2025, pp. 12838–12852.
[49] Z. Jiang, Z. Jin, G. He, PromptKeeper: Safeguarding system prompts for LLMs, in: EMNLP 2025, 2025, pp. 2712–2728.
[50] R. Dai, Y. Zhang, M. Pei, A. Li, T. Liu, X. Yang, B. Han, Safeguarding system prompts: A surrogate-based defense against injection attacks, OpenReview preprint (2024).
[51] Z. Zhuang, M.-I. Nicolae, H.-P. Wang, M. Fritz, ProxyPrompt: Securing system prompts against prompt extraction attacks, arXiv preprint arXiv:2505.11459 (2025).
[52] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al., Lora: Low-rank adaptation of large language models, in: ICLR, 2022.
[53] P. Ding, J. Kuang, D. Ma, X. Cao, Y. Xian, J. Chen, S. Huang, A wolf in sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily, in: NAACL, Vol. 1, 2024, p. 2136 – 2153.
[54] R. Xu, Y. Cai, Z. Zhou, R. Gu, H. Weng, Y. Liu, T. Zhang, W. Xu, H. Qiu, Course-correction: Safety alignment using synthetic preferences, in: EMNLP, 2024, p. 1622 – 1649.
[55] Z. Wang, F. Yang, L. Wang, P. Zhao, H. Wang, L. Chen, Q. Lin, K.-F. Wong, Self-guard: Empower the LLM to safeguard itself, in: ACL

NAACL, 2024, pp. 1648–1668.
[56] Y. Huang, S. Gupta, M. Xia, K. Li, D. Chen, Catastrophic jailbreak of open-source LLMs via exploiting generation, in: ICLR, 2024.
[57] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, D. Wagner, Jatmo: Prompt injection defense by task-specific finetuning, Lecture Notes in Computer Science 14982 (2024) 105 – 124.
[58] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al., Constitutional AI: Harmlessness from AI feedback, arXiv preprint arXiv:2212.08073 (2022).
[59] P. Röttger, H. R. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, D. Hovy, Xstest: A test suite for identifying exaggerated safety behaviours in large language models, arXiv preprint arXiv:2308.01263 (2023).
[60] Z. Zhang, J. Yang, P. Ke, F. Mi, H. Wang, M. Huang, Defending large language models against jailbreaking attacks through goal prioritization, in: ACL, Vol. 1, 2024, p. 8865 – 8887.
[61] A. Sheshadri, A. Ewart, P. Guo, A. Lynch, C. Wu, V. Hebbar, H. Sleight, A. C. Stickland, E. Perez, D. Hadfield-Menell, S. Casper, Latent adversarial training improves robustness to persistent harmful behaviors in LLMs, arXiv preprint arXiv:2407.15549 (2025).
[62] Y. Deng, W. Zhang, S. J. Pan, L. Bing, Multilingual jailbreak challenges in large language models, in: ICLR, 2024.
[63] T. Wu, S. Zhang, K. Song, S. Xu, S. Zhao, R. Agrawal, S. R. Indurthi, C. Xiang, P. Mittal, W. Zhou, Instructional segment embedding: Improving LLM safety with instruction hierarchy, arXiv preprint arXiv:2410.09102 (2024).
[64] S. Hiromi, H. Kinoshita, M. Yamada, T. Miura, Enhancing jailbreak resistance in large language models using model merge, in: IEEE SPW, 2025, pp. 111–117.
[65] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).
[66] S. Griffith, K. Subramanian, J. Scholz, C. L. Isbell, A. L. Thomaz, Policy shaping: Integrating human feedback with reinforcement learning, NeurIPS 26 (2013).
[67] A. Siththaranjan, C. Laidlaw, D. Hadfield-Menell, Distributional preference learning: Understanding and accounting for hidden context in RLHF, in: ICLR, 2024.
[68] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, A. Beutel, The instruction hierarchy: Training LLMs to prioritize privileged instructions, arXiv preprint arXiv:2404.13208 (2024).
[69] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, C. Finn, Direct preference optimization: Your language model is secretly a reward model, NeurIPS 36 (2023) 53728–53741.
[70] R. Eldan, M. Russinovich, Who's harry potter? Approximate unlearning in LLMs, arXiv preprint arXiv:2310.02238 (2023).
[71] N. Li, A. Pan, A. Gopal, S. Yue, D. Berrios, A. Gatti, J. D. Li, A.-K. Dombrowski, S. Goel, G. Mukobi, N. Helm-Burger, R. Lababidi, L. Justen, A. B. Liu, M. Chen, I. Barrass, O. Zhang, X. Zhu, R. Tamirisa, B. Bharathi, A. Herbert-Voss, C. B. Breuer, A. Zou, M. Mazeika, Z. Wang, P. Oswal, W. Lin, A. A. Hunt, J. Tienken-Harder, K. Y. Shih, K. Talley, J. Guan, I. Steneker, D. Campbell, B. Jokubaitis, S. Basart, S. Fitz, P. Kumaraguru, K. K. Karmakar, U. Tupakula, V. Varadharajan, Y. Shoshitaishvili, J. Ba, K. M. Esvelt, A. Wang, D. Hendrycks, The WMDP benchmark: Measuring and reducing malicious use with unlearning, in: ICML, Vol. 235, 2024, pp. 28525–28550.
[72] D. Huu-Tien, T. Pham, H. Thanh-Tung, N. Inoue, On effects of steering latent representation for large language model unlearning, in: AAAI Conference on Artificial Intelligence, 2025.
[73] A. Zou, L. Phan, J. Wang, D. Duenas, M. Lin, M. Andriushchenko, J. Z. Kolter, M. Fredrikson, D. Hendrycks, Improving alignment and robustness with circuit breakers, NeurIPS 37 (2024) 83345–83373.
[74] L. Schwinn, D. Dobre, S. Xhonneux, G. Gidel, S. Günnemann, Soft prompt threats: Attacking safety alignment and unlearning in open-source LLMs through the embedding space, NeurIPS 37 (2024) 9086–9116.
[75] T. Korbak, K. Shi, A. Chen, R. V. Bhalerao, C. Buckley, J. Phang, S. R. Bowman, E. Perez, Pretraining language models with human preferences, in: ICML, 2023, pp. 17506–17533.
[76] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, G. Irving, Fine-tuning language models from human preferences, arXiv preprint arXiv:1909.08593 (2019).
[77] A. B. Abacha, W.-w. Yim, Y. Fu, Z. Sun, M. Yetisgen, F. Xia, T. Lin, Medec: A benchmark for medical error detection and correction in clinical notes, arXiv preprint arXiv:2412.19260 (2024).
[78] W. Knight, OpenAI's CEO says the age of giant AI models is already over, https://www.wired.com/story/openai-ceo-sam-altman-the-age-of-giant-ai-models-is-already-over/, Medium (2023).
[79] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, et al., Prompt injection attack against LLM-integrated applications, arXiv preprint arXiv:2306.05499 (2023).
[80] R. Xu, Z. Qi, W. Xu, Preemptive answer "attacks" on chain-of-thought reasoning, in: ACL, 2024, p. 14708 – 14726.
[81] S. Ayachitula, C. Kundu, B. Mishra, Probing robustness of in-context learning in LLM classification predictions under label noise, in: IEEE TPS-ISA, 2024.
[82] C. Zheng, F. Yin, H. Zhou, F. Meng, J. Zhou, K.-W. Chang, M. Huang, N. Peng, On prompt-driven safeguarding for large language models, in: PMLR, Vol. 235, 2024, p. 61593 – 61613.
[83] Y. Zhou, Y. Han, H. Zhuang, K. Guo, Z. Liang, H. Bao, X. Zhang, Defending jailbreak prompts via in-context adversarial game, in: ACL EMNLP, 2024, p. 20084 – 20105.
[84] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, M. Zeng, Automatic prompt optimization with "gradient descent" and beam search, in: ACL EMNLP, 2023, p. 7957 – 7968.
[85] N. Jain, A. Schwarzschild, Y. Wen, G. Somepalli, J. Kirchenbauer, P.-y. Chiang, M. Goldblum, A. Saha, J. Geiping, T. Goldstein, Baseline defenses for adversarial attacks against aligned language models, arXiv preprint arXiv:2309.00614 (2023).
[86] L. L. Bochuan Cao, Yuanpu Cao, J. Chen, Defending against alignment-breaking attacks via robustly aligned LLM, in: ACL, 2024, p. 10542 – 10560.
[87] A. Robey, E. Wong, H. Hassani, G. J. Pappas, SmoothLLM: Defending large language models against jailbreaking attacks, arXiv preprint arXiv:2310.03684 (2024).
[88] J. Ji, B. Hou, Z. Zhang, G. Zhang, W. Fan, Q. Li, Y. Zhang, G. Liu, S. Liu, S. Chang, Advancing the robustness of large language models through self-denoised smoothing, in: ACL NAACL, Vol. 2, 2024, p. 246 – 257.
[89] S. Fan, Y. Yang, W. Huang, K. Ma, Y. Liu, Y. Zheng, Defense methods against multi-language and multi-intent LLM attacks, in: AHPCAI, Vol. 13403, 2024, pp. 423–430.
[90] C. Yung, H. M. Dolatabadi, S. Erfani, C. Leckie, Round trip translation defence against large language model jailbreaking attacks, in: Springer Nature Trends and Applications in Knowledge Discovery and Data Mining, 2025, pp. 286–297.
[91] Y. Wang, Z. Shi, A. Bai, C.-J. Hsieh, Defending LLMs against jailbreaking attacks via backtranslation, in: ACL, 2024, pp. 16031–16046.
[92] B. Pingua, D. Murmu, M. Kandpal, J. Rautaray, P. Mishra, R. K. Barik, M. J. Saikia, Mitigating adversarial manipulation in LLMs: a prompt-based approach to counter jailbreak attacks (Prompt-G), PeerJ Computer Science 10 (2024).
[93] B. Yi, T. Huang, S. Chen, T. Li, Z. Liu, Z. Chu, Y. Li, Probe before you talk: Towards black-box defense against backdoor unalignment for large language models, arXiv preprint arXiv:2506.16447 (2025).
[94] Z. Zhang, Q. Zhang, J. Foerster, PARDEN, can you repeat that? defending against jailbreaks via repetition, in: PMLR, Vol. 235, 2024, p. 60271 – 60287.
[95] K. Papineni, S. Roukos, T. Ward, W.-J. Zhu, Bleu: a method for automatic evaluation of machine translation, in: ACL, 2002, pp. 311–318.
[96] Y. Song, R. Liu, S. Chen, Q. Ren, Y. Zhang, Y. Yu, SecureSQL: Evaluating data leakage of large language models as natural language interfaces to databases, in: ACL EMNLP 2024, 2024, p. 5975 – 5990.
[97] Y. Zhang, N. Carlini, D. Ippolito, Effective prompt extraction from language models, arXiv preprint arXiv:2307.06865 (2024).
[98] Q. Liu, Z. Zhou, L. He, Y. Liu, W. Zhang, S. Su, Alignment-enhanced decoding: Defending jailbreaks via token-level adaptive refining of probability distributions, in: ACL EMNLP, 2024, p. 2802 – 2816.
[99] Z. Xu, F. Jiang, L. Niu, J. Jia, B. Y. Lin, R. Poovendran, SafeDecoding: Defending against jailbreak attacks via safety-aware decoding, in: ACL, Vol. 1, 2024, p. 5587 – 5605.
[100] P. Wang, D. Zhang, L. Li, C. Tan, X. Wang, M. Zhang, K. Ren, B. Jiang, X. Qiu, InferAligner: Inference-time alignment for harmless-

ness through cross-model guidance, in: EMNLP 2024, 2024, p. 10460 – 10479.
[101] A. Hasan, I. Rugina, A. Wang, Pruning for protection: Increasing jailbreak resistance in aligned LLMs without fine-tuning, in: BlackboxNLP, 2024, p. 417 – 430.
[102] M. Sun, Z. Liu, A. Bair, J. Z. Kolter, A simple and effective pruning approach for large language models, arXiv preprint arXiv:2306.11695 (2023).
[103] B. Wei, K. Huang, Y. Huang, T. Xie, X. Qi, M. Xia, P. Mittal, M. Wang, P. Henderson, Assessing the brittleness of safety alignment via pruning and low-rank modifications, in: PMLR, Vol. 235, 2024, p. 52588 – 52610.
[104] W. Zhao, Z. Li, Y. Li, Y. Zhang, J. Sun, Defending large language models against jailbreak attacks via layer-specific editing, in: ACL EMNLP, 2024, pp. 5094–5109.
[105] Q. Lan, AnujKaul, S. Jones, Prompt injection detection in LLM integrated applications, International Journal of Network Dynamics and Intelligence 4 (2) (2025) 100013.
[106] S. Irtiza, K. A. Akbar, A. Yasmeen, L. Khan, O. Daescu, B. Thuraisingham, LLM-Sentry: A model-agnostic human-in-the-loop framework for securing large language models, in: IEEE TPS-ISA, 2024, p. 245 – 254.
[107] K. Pokhrel, C. Sanin, M. K. H. Sakib, M. R. Islam, E. Szczerbicki, An adversarial machine learning approach on securing large language model with Vigil, an open-source initiative, Procedia Computer Science 246 (2024) 686–695.
[108] O. Muliarevych, Enhancing system security: LLM-driven defense against prompt injection vulnerabilities, in: IEEE TCSET, 2024, p. 420 – 423.
[109] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, et al., Llama guard: LLM-based input-output safeguard for human-AI conversations, arXiv preprint arXiv:2312.06674 (2023).
[110] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, N. Dziri, Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs, NeurIPS 37 (2024) 8093–8131.
[111] P. Gupta, L. Q. Yau, H. H. Low, I.-S. Lee, H. M. Lim, Y. X. Teoh, J. H. Koh, D. W. Liew, R. Bhardwaj, R. Bhardwaj, S. Poria, WalledEval: A comprehensive safety evaluation toolkit for large language models, in: ACL EMNLP, 2024, p. 397 – 407.
[112] Z. Yuan, Z. Xiong, Y. Zeng, N. Yu, R. Jia, D. Song, B. Li, RigorLLM: Resilient guardrails for large language models against undesired content, in: ICML, 2024.
[113] L. A. Tien, P. Van Huong, Optimizing transformer models for prompt jailbreak attack detection in AI assistant systems, in: VCRIS 2024, 2024.
[114] G. Alon, M. Kamfonas, Detecting language model attacks with perplexity, arXiv preprint arXiv:2308.14132 (2023).
[115] Z. Hu, G. Wu, S. Mitra, R. Zhang, T. Sun, H. Huang, V. Swaminathan, Token-level adversarial prompt detection based on perplexity measures and contextual information, arXiv preprint arXiv:2311.11509 (2023).
[116] Q. Chen, S. Yamaguchi, Y. Yamamoto, Defending against GCG jailbreak attacks with syntax trees and perplexity in LLMs, in: IEEE GCCE, 2024, p. 1411 – 1415.
[117] Y. Xie, M. Fang, R. Pi, N. Z. Gong, GradSafe: Detecting jailbreak prompts for LLMs via safety-critical gradient analysis, in: ACL, Vol. 1, 2024, p. 507 – 518.
[118] C. Qian, H. Zhang, L. Sha, Z. Zheng, Hsf: Defending against jailbreak attacks with hidden state filtering, in: Companion Proceedings of the ACM on Web Conference 2025, 2025, pp. 2078–2087.
[119] N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, J. Steinhardt, Eliciting latent predictions from transformers with the tuned lens, arXiv preprint arXiv:2303.08112 (2025).
[120] D. Lee, S. Xie, S. Rahman, K. Pat, D. Lee, Q. A. Chen, "Prompter Says"': A linguistic approach to understanding and detecting jailbreak attacks against large-language models, in: ACM LAMPS, 2024, p. 77 – 87.
[121] J. Wu, J. Deng, S. Pang, Y. Chen, J. Xu, X. Li, W. Xu, Legilimens: Practical and unified content moderation for large language model services, in: ACM CCS, 2024, p. 1151 – 1165.
[122] Y. Liu, J. Yu, H. Sun, L. Shi, G. Deng, Y. Chen, Y. Liu, Efficient detection of toxic prompts in large language models, in: IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 455–467.
[123] M. Kang, B. Li, R2-Guard: Robust reasoning enabled LLM guardrail via knowledge-enhanced logical reasoning, arXiv preprint arXiv:2407.05557 (2024).
[124] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, NeurIPS 35 (2022) 24824–24837.
[125] Y. Cao, N. Gu, X. Shen, D. Yang, X. Zhang, Defending large language models against jailbreak attacks through chain of thought prompting, in: NaNA, 2024, p. 125 – 130.
[126] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, J. Cohen, NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails, in: Conference on Empirical Methods in Natural Language Processing, 2023.
[127] National Cyber Security Centre, Machine learning principles, https://www.ncsc.gov.uk/collection/machine-learning-principles (2024).
[128] MITRE, Passive AI output obfuscation, https://atlas.mitre.org/mitigations/AML.M0002 (2025).
[129] MITRE, Restrict number of AI model queries, https://atlas.mitre.org/mitigations/AML.M0004 (2025).
[130] MITRE, Limit public release of information, https://atlas.mitre.org/mitigations/AML.M0000 (2025).
[131] MITRE, Limit model artifact release, https://atlas.mitre.org/mitigations/AML.M0001 (2023).
[132] S. Longpre, G. Yauney, E. Reif, K. Lee, A. Roberts, B. Zoph, D. Zhou, J. Wei, K. Robinson, D. Mimno, et al., A pretrainer's guide to training data: Measuring the effects of data age, domain coverage, quality, & toxicity, in: ACL NAACL, 2024, pp. 3245–3276.
[133] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, T. Goldstein, A watermark for large language models, in: ICML, 2023, pp. 17061–17084.
[134] European Union, EU AI act, https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689 (2024).
[135] H. Zhang, B. L. Edelman, D. Francati, D. Venturi, G. Ateniese, B. Barak, Watermarks in the sand: Impossibility of strong watermarking for generative models, arXiv preprint arXiv:2311.04378 (2023).
[136] V. S. Sadasivan, A. Kumar, S. Balasubramanian, W. Wang, S. Feizi, Can ai-generated text be reliably detected?, arXiv preprint arXiv:2303.11156 (2023).
[137] S. Omri, M. Abdelkader, M. Hamdi, Safetyrag: Towards safe large language model-based application through retrieval-augmented generation, Journal of Advances in Information Technology 16 (2025) 243–250.
[138] Z. Jin, S. Liu, H. Li, X. Zhao, H. Qu, Jailbreakhunter: a visual analytics approach for jailbreak prompts discovery from large-scale human-LLM conversational datasets, IEEE Transactions on Visualization and Computer Graphics (2025).
[139] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, D. Wagner, Promptshield: Deployable detection for prompt injection attacks, in: ACM CODASPY, 2024, pp. 341–352.
[140] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, C. H. Lampert, Can LLMs separate instructions from data? and what do we even mean by that?, arXiv preprint arXiv:2403.06833 (2024).
[141] W. Mai, G. Hong, P. Chen, X. Pan, B. Liu, Y. Zhang, H. Duan, M. Yang, You can't eat your cake and have it too: The performance degradation of LLMs with jailbreak defense, in: ACM WWW, 2025, pp. 872–883.
[142] H. Liu, H. Huang, X. Gu, H. Wang, Y. Wang, On calibration of LLM-based guard models for reliable content moderation, arXiv preprint arXiv:2410.10414 (2024).
[143] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramer, et al., Jailbreakbench: An open robustness benchmark for jailbreaking large language models, NeurIPS 37 (2024) 55005–55029.
[144] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, N. Inie, Garak: A framework for security probing large language models, arXiv preprint arXiv:2406.11036 (2024).
[145] Meta, Llama 2: open source, free for research and commercial use, https://www.llama.com/llama2/ (n.d.).
[146] Meta, Introducing Meta Llama 3: The most capable openly available LLM to date, https://ai.meta.com/blog/meta-llama-3/ (2024).
[147] Meta, The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation, https://ai.meta.com/blog/llama-4-multimodal-in

telligence/ (2025).
[148] J. Chim, J. Ive, M. Liakata, Evaluating synthetic data generation from user generated text, Computational Linguistics 51 (1) (2025) 191–233.
[149] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, Y. Gal, Ai models collapse when trained on recursively generated data, Nature 631 (8022) (2024) 755–759.
[150] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, Z. Sui, Large language models are not fair evaluators, in: ACL, 2024, pp. 9440–9450.
[151] L. Dietz, O. Zendel, P. Bailey, C. L. Clarke, E. Cotterill, J. Dalton, F. Hasibi, M. Sanderson, N. Craswell, Principles and guidelines for the use of LLM judges, in: ACM ICTIR, 2025, pp. 218–229.
[152] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-bench and Chatbot Arena, in: NeurIPS, 2023.
[153] J. Gallifant, M. Afshar, S. Ameen, Y. Aphinyanaphongs, S. Chen, G. Cacciamani, D. Demner-Fushman, D. Dligach, R. Daneshjou, C. Fernandes, et al., The TRIPOD-LLM reporting guideline for studies using large language models, Nature medicine 31 (1) (2025) 60–69.
[154] European Union, EU AI act, article 14: Human oversight, https://artificialintelligenceact.eu/article/14/ (2026).
[155] A. Sheshadri, A. Ewart, P. Guo, A. Lynch, C. Wu, V. Hebbar, H. Sleight, A. C. Stickland, E. Perez, D. Hadfield-Menell, S. Casper, Repository of Latent Adversarial Training, https://github.com/aengusl/latent-adversarial-training (2025).
[156] D. Chezelles, T. Le Sellier, S. O. Shayegan, L. K. Jang, X. H. Lù, O. Yoran, D. Kong, F. F. Xu, S. Reddy, Q. Cappart, et al., The browsergym ecosystem for web agent research, arXiv preprint arXiv:2412.05467 (2024).
[157] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: NeurIPS, Vol. 37, 2024.
[158] T. Liu, Y. Zhang, Z. Zhao, Y. Dong, G. Meng, K. Chen, Making them ask and answer: Jailbreaking large language models in few queries via disguise and reconstruction, in: USENIX, 2024, pp. 4711–4728.
[159] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. Forsyth, D. Hendrycks, Harmbench: a standardized evaluation framework for automated red teaming and robust refusal, in: ICML, 2024.
[160] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, A. Karbasi, Tree of attacks: Jailbreaking black-box LLMs automatically, in: NeurIPS, Vol. 37, 2024.
[161] Y. Zeng, H. Lin, J. Zhang, D. Yang, R. Jia, W. Shi, How johnny can persuade LLMs to jailbreak them: Rethinking persuasion to challenge AI safety by humanizing LLMs, in: ACL, 2024, pp. 14322–14350.
[162] J. Yu, X. Lin, Z. Yu, X. Xing, GPTFUZZER: Red teaming large language models with auto-generated jailbreak prompts, arXiv preprint arXiv:2309.10253 (2023).
[163] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Anati, V. Tagliabue, A. Kost, C. Carnahan, J. Boyd-Graber, Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition, in: ACL EMNLP, 2023, pp. 4945–4977.
[164] J. Piet, M. Alrashed, C. Sitawarin, S. Chen, Z. Wei, E. Sun, B. Alomair, D. Wagner, Repository of Jatmo, https://github.com/wagner-group/prompt-injection-defense (2024).
[165] Y. Yuan, W. Jiao, W. Wang, J.-t. Huang, P. He, S. Shi, Z. Tu, GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher, arXiv preprint arXiv:2308.06463 (2023).
[166] B. Peng, C. Li, P. He, M. Galley, J. Gao, Instruction tuning with GPT-4, arXiv preprint arXiv:2304.03277 (2023).
[167] Y. Deng, W. Zhang, S. J. Pan, L. Bing, Repository of SelfDefense, https://github.com/DAMO-NLP-SG/multilingual-safety-for-LLMs (2024).
[168] P. Ding, J. Kuang, D. Ma, X. Cao, Y. Xian, J. Chen, S. Huang, Repository of ReNeLLM, https://github.com/NJUNLP/ReNeLLM (2024).
[169] Y. Liu, G. Deng, Z. Xu, Y. Li, Y. Zheng, Y. Zhang, L. Zhao, T. Zhang, K. Wang, Y. Liu, Jailbreaking chatgpt via prompt engineering: An empirical study, arXiv preprint arXiv:2305.13860 (2023).
[170] Z. Zhang, J. Yang, P. Ke, F. Mi, H. Wang, M. Huang, Repository of Goal Prioritization, https://github.com/thu-coai/JailbreakDefense_GoalPriority (2024).
[171] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al., Training a helpful and harmless assistant with reinforcement learning from human feedback, arXiv preprint arXiv:2204.05862 (2022).
[172] A. Siththaranjan, C. Laidlaw, D. Hadfield-Menell, Repository of "distributional preference learning"', https://github.com/cassidylaidlaw/hidden-context.git (2024).
[173] A. Zou, L. Phan, J. Wang, D. Duenas, M. Lin, M. Andriushchenko, J. Z. Kolter, M. Fredrikson, D. Hendrycks, Repository of Circuit Breakers, https://github.com/GraySwanAI/circuit-breakers (2024).
[174] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, P. Henderson, Fine-tuning aligned language models compromises safety, even when users do not intend to!, arXiv preprint arXiv:2310.03693 (2023).
[175] X. Wang, S. Zhu, X. Cheng, Repository of SSD, https://github.com/k-k1w-w1x-x/Speculative-Safety-Aware-Decoding (2025).
[176] S. Lin, J. Hilton, O. Evans, TruthfulQA: Measuring how models mimic human falsehoods, in: ACL, 2022, pp. 3214–3252.
[177] P. Wang, D. Zhang, L. Li, C. Tan, X. Wang, M. Zhang, K. Ren, B. Jiang, X. Qiu, Repository of InferAligner, https://github.com/Jihuai-wpy/InferAligner (2024).
[178] Z. Wei, Y. Wang, A. Li, Y. Mo, Y. Wang, Jailbreak and guard aligned language models with only few in-context demonstrations, arXiv preprint arXiv:2310.06387 (2023).
[179] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al., Training verifiers to solve math word problems, arXiv preprint arXiv:2110.14168 (2021).
[180] Y. Dubois, C. X. Li, R. Taori, T. Zhang, I. Gulrajani, J. Ba, C. Guestrin, P. S. Liang, T. B. Hashimoto, Alpacafarm: A simulation framework for methods that learn from human feedback, NeurIPS 36 (2023) 30039–30069.
[181] Q. Liu, Z. Zhou, L. He, Y. Liu, W. Zhang, S. Su, Repository of AED, https://github.com/GIGABaozi/AED (2024).
[182] X. Li, Z. Zhou, J. Zhu, J. Yao, T. Liu, B. Han, DeepInception: Hypnotize large language model to be jailbreaker, arXiv preprint arXiv:2311.03191 (2023).
[183] B. Deng, W. Wang, F. Feng, Y. Deng, Q. Wang, X. He, Attack prompt generation for red teaming and defending large language models, arXiv preprint arXiv:2310.12505 (2023).
[184] Z. Xu, F. Jiang, L. Niu, J. Jia, B. Y. Lin, R. Poovendran, Repository of SafeDecoding, https://github.com/uw-nsl/SafeDecoding (2024).
[185] W. Zhao, Z. Li, Y. Li, Y. Zhang, J. Sun, Repository of LED, https://github.com/ledllm/ledllm.git (2024).
[186] C. Zheng, F. Yin, H. Zhou, F. Meng, J. Zhou, K.-W. Chang, M. Huang, N. Peng, Repository of DRO, https://github.com/chujiezheng/LLM-Safeguard (2024).
[187] Y. Zhou, Y. Han, H. Zhuang, K. Guo, Z. Liang, H. Bao, X. Zhang, Repository of "defending jailbreak prompts via in-context adversarial game"', https://github.com/YujunZhou/In-Context-Adversarial-Game (2024).
[188] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, C. Potts, Recursive deep models for semantic compositionality over a sentiment treebank, in: ACL EMNLP, 2013, pp. 1631–1642.
[189] Microsoft, Repository of ProTeGi, https://github.com/microsoft/LMOps/tree/main/prompt_optimization (2025).
[190] Y. Xie, J. Yi, J. Shao, J. Curl, L. Lyu, Q. Chen, X. Xie, F. Wu, Repository of Self-reminder, https://github.com/yjw1029/Self-Reminder/tree/master (2023).
[191] Z. Zhou, Q. Wang, M. Jin, J. Yao, J. Ye, W. Liu, W. Wang, X. Huang, K. Huang, Mathattack: Attacking large language models towards math solving ability, in: AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 19750–19758.
[192] C. Yung, H. M. Dolatabadi, S. Erfani, C. Leckie, Repository of RTT, https://github.com/Cancanxxx/Round_Trip_Translation_Defence (2024).
[193] Y. Wang, Z. Shi, A. Bai, C.-J. Hsieh, Repository of Backtranslation, https://github.com/YihanWang617/llm-jailbreaking-defense (2024).
[194] J. Ji, B. Hou, Z. Zhang, G. Zhang, W. Fan, Q. Li, Y. Zhang, G. Liu, S. Liu, S. Chang, Repository of SelfDenoise, https://github.com/UCSB-NLP-Chang/SelfDenoise (2024).
[195] Y. Dubois, B. Galambosi, P. Liang, T. B. Hashimoto, Length-controlled alpacaeval: A simple way to debias automatic evaluators, arXiv preprint arXiv:2404.04475 (2024).
[196] L. Jiang, K. Rao, S. Han, A. Ettinger, F. Brahman, S. Kumar,

N. Mireshghallah, X. Lu, M. Sap, Y. Choi, et al., Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models, NeurIPS 37 (2024) 47094–47165.
[197] S. Xiang, A. Zhang, Y. Cao, Y. Fan, R. Chen, Repository of "beyond surface-level patterns"', https://github.com/ShiyuXiang77/EDDF (2025).
[198] B. Yi, T. Huang, S. Chen, T. Li, Z. Liu, Z. Chu, Y. Li, Repository of BEAT, https://github.com/clearloveclearlove/BEAT (2025).
[199] M. Andriushchenko, F. Croce, N. Flammarion, Jailbreaking leading safety-aligned LLMs with simple adaptive attacks, arXiv preprint arXiv:2404.02151 (2024).
[200] Z. Liao, H. Sun, Amplegcg: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed LLMs, arXiv preprint arXiv:2404.07921 (2024).
[201] A. Robey, E. Wong, H. Hassani, G. J. Pappas, Repository of SmoothLLM, https://github.com/arobey1/smooth-llm (2024).
[202] Z. Zhang, Q. Zhang, J. Foerster, Repository of PARDEN, https://github.com/Ed-Zh/PARDEN (2024).
[203] J. Yi, Self-reminder data, https://github.com/yjw1029/Self-Reminder-Data/blob/master/data/jailbreak_prompts.csv (2023).
[204] V. Zuo, Forbidden question set, https://github.com/verazuo/jailbreak_llms/blob/main/data/forbidden_question/forbidden_question_set.csv (2024).
[205] B. Pingua, D. Murmu, M. Kandpal, J. Rautaray, P. Mishra, R. K. Barik, M. J. Saikia, Repository of Prompt-G, https://zenodo.org/records/13501821 (2024).
[206] Y. Song, R. Liu, S. Chen, Q. Ren, Y. Zhang, Y. Yu, Repository of SecureSQL, https://github.com/JacobiSong/SecureSQL (2024).
[207] Z. Jiang, Z. Jin, G. He, Repository of PromptKeeper, https://github.com/SamuelGong/PromptKeeper (2025).
[208] Y. Zhang, N. Carlini, D. Ippolito, Repository of "effective prompt extraction from language models", https://github.com/y0mingzhang/prompt-extraction (2024).
[209] FAR.AI, Tuned lens repository, https://github.com/AlignmentResearch/tuned-lens (2024).
[210] M. F. A. Sayeedi, M. Bin Hossain, M. K. Hassan, S. Afrin, M. M. S. Hossain, M. S. Hossain, JailbreakTracer: Explainable detection of jailbreaking prompts in LLMs using synthetic data generation, IEEE Access 13 (2025) 123708–123723.
[211] M. F. A. Sayeedi, M. Bin Hossain, M. K. Hassan, S. Afrin, M. M. S. Hossain, M. S. Hossain, Repository of JailbreakTracer, https://github.com/faiyazabdullah/JailbreakTracer/tree/main (2025).
[212] Z. Lin, Z. Wang, Y. Tong, Y. Wang, Y. Guo, Y. Wang, J. Shang, Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation, arXiv preprint arXiv:2310.17389 (2023).
[213] Z. Yuan, Z. Xiong, Y. Zeng, N. Yu, R. Jia, D. Song, B. Li, Repository of RigorLLM, https://github.com/eurekayuan/RigorLLM.git (2024).
[214] K. Pokhrel, C. Sanin, M. K. H. Sakib, M. R. Islam, E. Szczerbicki, Repository of Vigil, https://github.com/deadbits/vigil-llm.git (2023).
[215] P. Gupta, L. Q. Yau, H. H. Low, I.-S. Lee, H. M. Lim, Y. X. Teoh, J. H. Koh, D. W. Liew, R. Bhardwaj, R. Bhardwaj, S. Poria, Repository of WalledEval, https://github.com/walledai/walledeval?tab=readme-ov-file#flow-1-prompt-benchmarking (2024).
[216] B. L. Mintong Kang, Repository of R2-Guard, https://github.com/kangmintong/R-2-Guard (2024).
[217] J. Ji, M. Liu, J. Dai, X. Pan, C. Zhang, C. Bian, B. Chen, R. Sun, Y. Wang, Y. Yang, Beavertails: Towards improved safety alignment of LLM via a human-preference dataset, NeurIPS 36 (2023) 24678–24704.
[218] J. Wu, J. Deng, S. Pang, Y. Chen, J. Xu, X. Li, W. Xu, Repository of Legilimens, https://github.com/lin000001/Legilimens (2025).
[219] S. Irtiza, K. A. Akbar, A. Yasmeen, L. Khan, O. Daescu, B. Thuraisingham, Repository of LLM-sentry, https://github.com/Saquibirtiza/LLM-Sentry (2024).
[220] Y. Xie, M. Fang, R. Pi, N. Z. Gong, Repository of Gradsafe, https://github.com/xyq7/GradSafe (2024).
[221] S. Gehman, S. Gururangan, M. Sap, Y. Choi, N. A. Smith, RealToxicityPrompts: Evaluating neural toxic degeneration in language models, arXiv preprint arXiv:2009.11462 (2020).
[222] Y. Zeng, Y. Wu, X. Zhang, H. Wang, Q. Wu, Repository of AutoDefense (2024).
URL https://github.com/XHMY/AutoDefense
[223] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, N. Dziri, WildGuardMix, https://huggingface.co/datasets/allenai/wildguardmix (2024).
[224] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert, Y. Choi, N. Dziri, Repository of WildGuard, https://github.com/allenai/wildguard (2025).
[225] R. D. Jaramillo, ChatGPT jailbreak prompts, https://huggingface.co/datasets/rubend18/ChatGPT-Jailbreak-Prompts (2023).
[226] M. Zhang, X. Pan, M. Yang, Jade: A linguistics-based safety evaluation platform for LLM, arXiv preprint arXiv:2311.00286 (2023).
[227] NVIDIA, Repository of NeMo-Guardrails, https://github.com/NVIDIA/NeMo-Guardrails (2025).
[228] R. Kuditipudi, J. Thickstun, T. Hashimoto, P. Liang, Repository of "robust distortion-free watermarks for language models"', https://github.com/jthickstun/watermark (2024).
[229] J. Zhang, Y. Zhao, M. Saleh, P. Liu, Pegasus: Pre-training with extracted gap-sentences for abstractive summarization, in: ICML, 2020, pp. 11328–11339.
[230] K. Krishna, Y. Song, M. Karpinska, J. Wieting, M. Iyyer, Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense, NeurIPS 36 (2023) 27469–27500.
[231] X. Xu, Y. Yao, Y. Liu, Repository of "learning to watermark LLM-generated text via reinforcement learning", https://github.com/xiaojunxu/learning-to-watermark-llm (2024).
[232] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, Journal of machine learning research 21 (140) (2020) 1–67.
[233] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, T. Goldstein, Repository of KGW, https://github.com/jwkirchenbauer/lm-watermarking (2024).
[234] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents, arXiv preprint arXiv:2403.02691 (2024).
[235] Y. Wu, F. Roesner, T. Kohno, N. Zhang, U. Iqbal, Repository of IsolateGPT, https://github.com/llm-platform-security/SecGPT (2024).
[236] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Repository of CaMeL, https://github.com/google-research/camel-prompt-injection (2025).
[237] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, D. Wagner, PromptShield Dataset, https://huggingface.co/datasets/hendzh/PromptShield (2025).
[238] H. Liu, H. Huang, X. Gu, H. Wang, Y. Wang, Repository of "on calibration of LLM-based guard models for reliable content moderation"', https://github.com/Waffle-Liu/calibration_guard_model (2025).
[239] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramer, et al., Repository of Jailbreakbench, https://github.com/JailbreakBench/jailbreakbench (2024).
[240] NVIDIA, Repository of Garak, https://github.com/NVIDIA/garak (2024).
[241] X. Liu, N. Xu, M. Chen, C. Xiao, Repository of AutoDAN, https://github.com/SheltonLiu-N/AutoDAN (2024).
[242] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, M. Fredrikson, Repository of GCG, https://github.com/llm-attacks/llm-attacks (2023).
[243] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, E. Wong, Repository of PAIR, https://github.com/patrickrchao/JailbreakingLLMs (2023).

Appendix A. Overview of the selected works

To facilitate quick reference to specific works, this section presents tables that catalog the selected studies (Tables A.5, A.6, A.7, A.8, A.9, A.10, and A.11). For each work, the tables specify the reference to its source code, if open-source, or N/A otherwise. They also denote whether the proposed intervention is model-agnostic (✓) or requires access to the model parameters or architecture (✗). Moreover, the works are grouped according to the taxonomy. Some fit into multi-

ple categories. In such cases, we classify them according to the category to which we consider they contribute the most.

Most tables include the column "attack mitigation," which displays at least some of the relevant results reported in the corresponding work. The preferred metric is the attack success rate (ASR). Studies using alternative but equivalent metrics are presented as ASR. The standard notation is:

<metric> (attack): <result_before_intervention> → <result_after_intervention> (model)

The attack field is omitted when a work employs a single attack, as indicated in the "attack" column. In some cases, the model or baseline results are not reported in the original work and are therefore not shown. The reported results and models are not necessarily exhaustive. When applicable, the displayed value represents the average of the provided metrics across multiple tasks or scenarios. Percentage symbols are omitted.

Table A.5: Overview of training-time interventions.

Ref.	Year	Method	Description	Model-agnostic	Attack mitigation	Attack	Source code
Targeted LAT[61]	2025	Safety training (post-training)	Adversarial training that operates directly in the latent space of the transformer's internal activations	✗	ASR (PAIR): 17.7 → 2.5 (llama2-7b-chat)	PAIR[26], GCG/AdvBench [19]	[155]
ARLAS [43]	2025	Safety training (post-training)	Reinforcement learning with attack model and agent "warmmed up" by imitation learning	✗	ASR (AgentDojo): 6.3 → 5.4 (gemma3-12b), 1.6 → 1.4 (qwen3-14b)	BrowserGym [156], AgentDojo [157]	N/A
LatentGuard [44]	2025	Safety training (post-training)	Combines fine-tuning with structured variational autoencode	✗	ASR: 98.3 → 2.7 (qwen3-8b), 99 → 0.6 (mistral-7b)	GCG/AdvBench [19], DRA [158], Harmbench[159], TAP [160], PAP [161]	N/A
Model Merge [64]	2025	Safety training (post-training)	Integrates security-focused models into LLMs with model merge	✗	ASR: 91.9 → 78.4 (mistral-7b)	GCG/AdvBench [19]	N/A
Adversarial Tuning [42]	2024	Safety training (post-training)	Adversarial training using token-level augmentation and prompt-level adversarial refinement	✗	ASR: 51.2 → 0 (vicuna-13b), 9.6 → 0 (llama-2-7b)	GCG/AdvBench [19], TAP [160], AutoDAN [47], PAIR [26], gptFuzzer [162]	N/A
Jatmo [57]	2024	Safety training (post-training)	Fine-tunes non-instruction-tuned models with safe teacher model	✗	ASR: 87 → ∼0 (gpt-3.5-turbo)	HackAPrompt [163]	[164]
[56]	2024	Safety training (post-training)	Alignment with varied generation settings, as existing fine-tuning uses default decoding configuration	✗	ASR: 95 → 65 (llama-2)	GCG/AdvBench [19], own	N/A
[54]	2024	Safety training (post-training)	Fine-tuning whit course-correction to reduce jailbreak ASR while preserving utility	✗	ASR (GCG): 71 → 38.6 (llama2-chat), 66.7 → 46 (qwen2)	GCG/AdvBench [19], CipherChat [165], AutoDAN [47], PAIR [26]	[54]
Self-Guard [55]	2024	Safety training (post-training)	Model trained to self-review its responses	✗	ASR: 57.8 → 7.15 (vicuna-v1.5)	Alpaca [166]	N/A
Self-Defense [62]	2024	Safety training (post-training)	Multilingual LLM-generated data to reduce harmful outputs	✗	ASR: 80.9 → 60 (gpt-3.5-turbo-0613)	Own (MultiJail) [167]	[167]
ReNeLLM [53]	2024	Safety training (post-training)	Combines system prompt defense, fine-tuning, and harm classification	✗	ASR : 96 → 1.6 (gpt-4), 97.9 → 0 (claude-2)	Own [168]	[168]
Goal Prioritization [60]	2024	Safety training (post-training)	Trains LLMs to distinguish and prioritize safety over helpfulness	✗	ASR: 48.3 → 3.1 (gpt-4), 21 → 2.5 (llama2-13b-chat)	GCG/AdvBench [19], Jailbroken [23], gptFuzzer [162], [169]	[170]
DPL [67]	2024	Safety training (post-training)	Learns distribution of preferences for safer choices with high variance	✗	ASR: 25 → 13 (llama-2-7b)	HH-RLHF [171]	[172]
[75]	2023	Safety training (pre-training)	Conditions token generation on reward-model preference scores, substantially reducing undesirable outputs	✗	ASR: 27 → 2.5 (gpt-2-small)	N/A	N/A
[72]	2025	Unlearning	Adds noise to internal representations to unlearn targeted data while preserving overall performance	✗	Custom metrics	Own [72]	[72]
RMU [71]	2024	Unlearning	Perturbs activations on harmful data	✗	ACC on unlearned topics: 53.9 → 29.7 (zephyr-7b), 62.5 → 29.9 (yi-34b)	N/A	[71]
[73]	2024	Unlearning	Reroutes internal representations to safe or refusal states to block harmful outputs	✗	ASR: 39 → 2 (llama-3-8b-instruct)	Harmbench [159]	[173]

Table A.6: Overview of deployment-time interventions.

Ref.	Year	Method	Description	Model-agnostic	Attack mitigation	Attack	Source code
SSD [48]	2025	Decoding	Constructs the output probability space using two models, one, smaller, being rigorously aligned	✗	ASR: 33.9 → 6.4 (llama2-7b), 67.6 → 11.9 (vicuna-7b)	Harmful HEx-PHI [174]	[175]
InferAligner [100]	2024	Decoding	Uses Safety Steering Vectors to dynamically align the model at inference time	✗	ASR: 37.27 → 0.07 (llama2-7b)	GCG/AdvBench [19], TruthfulQA [176]	[177]
AED [98]	2024	Decoding	Dynamically refines token predictions using a competitive index	✗	ASR (AutoDAN): 26 → 10 (llama3-7b-instruct), 78 → 66 (gemma-1.1-7b-it)	GCG/AdvBench [19], AutoDAN [47], ICD [178], Refusal Supression [23], GMS8K [179], Alpaca [180]	[181]
SafeDecoding [99]	2024	Decoding	Utilizes dual-model decoding with temperature control and response normalization	✗	ASR (Advbench): 8 → 0 (vicuna); ASR (PAIR): 88 → 4 (vicuna)	HEx-PHI [174], Advbench [19], AutoDAN [47], PAIR [26], DeepInception [182], SAP30 [183], gptFuzzer [162]	[184]
[101]	2024	Pruning	Uses WANDA pruning to increase refusal rates and reduces ASR	✗	ASR: 40 → 0 (llama-2, base with 30% pruning)	[101], GCG/AdvBench [19]	N/A
LED [104]	2024	Pruning	Reveals LLM safety layers by pruning and testing responses	✗	ASR (PAIR): 85 → 8.1 (mistral-7b), 2.1 → 0 (llama2-7b)	GCG/AdvBench [19], AutoDAN [47], PAIR [26]	[185]
DRO [82]	2024	Prompt instruction and formatting	Optimizes the system safety prompt through fine-tuning	✗	ASR: 13.3 → 3.1 (llama-2-chat)	N/A	[186]
ICAG [83]	2024	Prompt instruction and formatting	Utilizes an in-context "adversarial game" to iteratively refines safety instructions	✓	ASR: 40.7 → 0.5 (gpt-3.5), 64.5 → 50.6 (vicuna)	GCG/AdvBench [19], Self-Reminder [40]	[187]
[81]	2024	Prompt instruction and formatting	Inserts a textual alert in the prompt to warn the model about possible incorrect labels	✓	ASR: 40 → 15 (gpt-4), 60 → 25 (gemini-pro)	SST-2 [188]	N/A
ProTeGi [84]	2023	Prompt instruction and formatting	Iteratively refines prompts using malicious examples for contrast	✓	F1: 0.65 → 0.85 (gpt-3.5-turbo)	N/A	[189]
Self-Reminder [40]	2023	Prompt instruction and formatting	Adds instructions before and after the user prompt to reinforce ethical standards	✓	ASR: 20 → 4.7 (chatgpt-4), 14.2 → 5 (llama-2)	Own [190]	[190]
RTT [90]	2025	Input modification	Translates prompts through a chain of languages and back to disrupt malicious instructions	✓	ASR (PAIR): 98 → 26 (vicuna)	MathAtack [191], PAIR [26]	[192]
[89]	2024	Input modification	Translates the input, breaks it into separate intents, and reviews each generated response	✓	ASR: 92 → 6	N/A	N/A
Backtranslation [91]	2024	Input modification	Infers the original prompt based on the output and passes it to the LLM	✓	ASR: 6 → 0 (GCG/gpt-3.5-turbo), 36 → 2 (PAIR/llama-2-13b)	GCG/AdvBench [19], AutoDAN [47], PAIR [26]	[193]
SelfDenoise [88]	2024	Input modification	Replaces tokens to circumvent jailbreak prompts	✓	ASR: 93 → 0 (vicuna-1.5-13b)	GCG/AdvBench [19], PAIR [26]	[194]
[85]	2023	Input modification	Paraphrases and retokenizes the original prompt	✓	ASR (AlpacaEval): 79 → 5 (vicuna-7b-v1.1)	GCG/AdvBench [19], AlpacaEval [195]	N/A
[39]	2025	Aggregating output form multiple prompts	Employs a vector database of adversarial prompt intents to filter inputs	✓	ASR: 80.3 → 5.7 (avg: deepseek, llama, qwen, gpt)	In-the-wild [25], human-red-teaming [196]	[197]
BEAT [93]	2025	Aggregating output form multiple prompts	Detects and deactivates backdoors during inference by analyzing distortions in output distributions	✓	AUROC (Advbench): 99.7	Advbench [19]	[198]
SmoothLLM [87]	2024	Aggregating output from multiple prompts	Perturbs several prompt variants and aggregate their outputs	✓	ASR (GCG): 5.6 → 0.8 (gpt-4), 51 → 0.1 (llama2)	GCG/AdvBench [19], PAIR [26], RandomSearch [199], AmpleGCC [200]	[201]
PARDEN [94]	2024	Aggregating output from multiple prompts	Adds safe and harmful examples to the prompt and compare results with the original output	✓	AUROC: 0.66 → 0.96 (llama2)	GCG/AdvBench [19], AutoDAN [47]	[202]
Prompt-G [92]	2024	Aggregating output from multiple prompts	Detects similarities with DAN prompts and respective outputs	✓	ASR/FP: 2.1/6 (llama-2-13b-chat)	Own [203, 204]	[205]
SecureSQL [96]	2024	Aggregating output from multiple prompts	Employs a chain-of-thought LLM to review SQL commands generated by another LLM	✓	Limited effectiveness (llama, qwen, mixtral, gpt, glm)	N/A	[206]
RA-LLM [86]	2024	Aggregating output from multiple prompts	Utilizes robust alignment check (RAC) based on stochastic removal of input text segments	✓	ASR: 98.7 → 10.7 (vicuna-7b)	GCG/AdvBench [19], AutoDAN [47], TAP [160]	N/A

Table A.7: Overview of deployment-time interventions (part 2).

Ref.	Year	Method	Description	Model-agnostic	Attack mitigation	Attack	Source code
ProxyPrompt [51]	2025	Prompt stealing prevention	Replaces the system prompt with a proxy in the embedding space	✓	ASR: 99 → 36 (llama-3.1-8b-instruct)	Own	N/A
PromptKeeper [49]	2025	Prompt stealing detection	Detects leakage based on probability and, if detected, regenerates repose with a dummy prompt	✓	ASR: 91 → 73.1 (llama3.1-8b-Instruct)	Own	[207]
SurF [50]	2024	Prompt stealing detection	Simulates interaction with surrogate prompts to detect leakage	✓	ASR: 38.5 → 19.9	[97]	N/A
[97]	2024	Prompt stealing detection	Filters output for n-grams that match the system prompt	✓	ASR: 95.2 → 72.3 (llama-2-70b-chat)	Own [208]	[208]
Constitutional Classifier [37]	2025	Input/output filtering	Classifies inputs and outputs using a model trained on allowed and forbidden categories	✓	ASR: 86 → 5 (claude-3.5-haiku)	N/A	N/A
Tuned Lens [119]	2025	Input/output filtering	Interpretability-based mitigation	✗	ASR: 60 → 0 in some tasks (pythia-12b)	HH-RLHF [171]	[209]
[105]	2025	Input/output filtering	Combines heuristic filtering, embeddings, and finetuned BERT to filter prompts	✓	F1: ∼1 (bert)	Own	N/A
JailbreakTracer	2025	Input/output filtering	Uses XAI techniques to explain classifier decisions	✓	ASR: 8.1 (llama-3.2), 15.5 (gpt-3.5-turbo)	Own	[211]
HSF [118]	2025	Input/output filtering	Analyzes hidden states during inference to identify and block harmful inputs before generating responses	✗	ASR: 98 → 0 (mistral-v0.2), 10 → 0 (llama2-7b)	AutoDAN [47]	N/A
RigorLLM [112]	2024	Input/output filtering	Combines prompt enhancement, adversarial data creation, and prediction aggregation for content moderation	✓	F1 (ToxicChat): 74.6	GCG/AdvBench [19], AutoDAN [47], PAIR [26], ToxicChat [212]	[213]
Vigil [107]	2024	Input/output filtering	Multi-filter encompassing vector database search, rule filtering, and model based classification	✓	ASR: 11 (gpt-3.5-turbo)	Own [214]	[214]
Prompter Says[120]	2024	Input/output filtering	Uses linguistic analysis to detect malicious prompts through six defined classifications (rubrics)	✓	ASR: 88.5 using logistic regression	N/A	N/A
[116]	2024	Input/output filtering	Employs syntax trees and perplexity to detect jailbreaks	✓	ASR: 0.7	GCG/AdvBench [19]	N/A
WalledEval [111]	2024	Input/output filtering	Detects harmful outputs with lightweight classifier models	✓	ASR: 69.7 → 7.2	XSTest [59], SGXSTest, HIXSTest	[215]
R^2-Guard [123]	2024	Input/output filtering	Combines data-driven learning and probabilistic logical reasoning to detect and moderate unsafe content	✓	ASR: 1.3	GCG/AdvBench [19]	[216]
[113]	2024	Input/output filtering	Fine-tunes models as external binary classifiers to block jailbreak prompts before they reach the target LLM	✓	ASR: 45.8 → 2.1 (gpt-4)	[169]	N/A
Legilimens [121]	2024	Input/output filtering	Classifies features produced by an LLM over a prompt using a neural network	✓	ASR: 2.7 (llama2-7b), 6.1 (vicuna-7b-v1.5)	BeaverTails [217]	[218]
LLM-sentry [106]	2024	Input/output filtering	Employs RAG to compare the input to a knowledge base of harmful prompts	✓	ASR: 3	Own	[219]
[108]	2024	Input/output filtering	Filters inputs based on intention analysis and fuzzy search	✓	ASR: 32.2 → 11.2 (gpt-4), 43.4 → 24.2 (gpt-3)	N/A	N/A
GradSafe [117]	2024	Input/output filtering	Filters prompts based on gradient similarity obtained with unsafe prompts	✗	F1: 0.24 → 0.75 (llama-2-7b-chat-hf)	ToxicChat [212]	[220]
Toxic-Detector [122]	2024	Input/output filtering	Detects harmful prompts by comparing inputs with toxic concepts	✓	F1: 0.998 (llama3-70b), 0.964 (gemma2-9b)	RealToxicityPrompts [221]	N/A
Autodefense [41]	2024	Input/output filtering	Analyzes an LLM's output to detect harmful content and overrides unsafe responses before they reach the user	✓	ASR: 55.74 → 7.95 (gpt-3.5-turbo)	AutoDAN [47]	[222]
WildGuard [110]	2024	Input/output filtering	Detects harmful, unsafe responses, and improper refusals based on multitask LLM safety filtering	✓	F1: 86.1	Own (wildguardmix) [223]	[224]
[115]	2024	Input/output filtering	Detects token-level attacks using context-aware perplexity	✓	F1: 1 (gpt-2-1.5b), 1 (llama2-7b)	GCG/AdvBench [19]	N/A
[114]	2023	Input/output filtering	Identifies attacks based on perplexity using a Light-GBM classifier	✓	F1 (GCG): 0.99 (gpt-2)	GCG/AdvBench [19], Human-crafted dataset [225]	N/A
Llama-Guard [109]	2023	Input/output filtering	Filters user prompts and model with an instruction-tuned	✓	AUPRC: 84.3	ToxicChat [212]	N/A

Table A.8: Overview of deployment-time interventions (part 3).

Ref.	Year	Method	Description	Model-agnostic	Attack mitigation	Attack	Source code
Problem Restatement [80]	2024	Self-reflection	Includes instruction to restate the problem in user prompt	✓	ASR: 30 → 24 (gpt-3.5-turbo-1106)	Own	N/A
[125]	2024	Self-reflection	Props model to self-check its outputs using chain of thought (Figure 11)	✓	ASR: 49.1 → 0.1 (llama-2-7b-chat)	DAN [25], JADE [226]	N/A
NeMo Guardrails [126]	2023	Self-reflection	Programmable guardrails	✓	ASR: 7 → 1; FP: 2 (gpt-3.5-turbo)	Anthropic's datasets: HH-RLHF [171], [27]	[227]

Table A.9: Overview of indirect mitigations.

Ref.	Year	Method	Description	Model-agnostic	Target models	Attack	Source code
[45]	2024	Watermarking	Uses sequence alignment to embed a distortion-free key sequence into text	✗	OPT-1.3B, LLaMA-7B	Own	[228]
[46]	2024	Watermarking	Trains a generator and detector to inject and detect watermarks	✗	OPT-1.3B, llama2-7B	Pegasus [229], DIPPER [230]	[231]
KGW [133]	2023	Watermarking	Watermarks LLMs' outputs with statistically biased tokens	✗	OPT-1.3B	T5-Large [232]	[233]

Table A.10: Overview of mitigations against indirect prompt injection.

Ref.	Year	Description	Model-agnostic	Attack mitigation	Attack	Source code
Spotlighting [34]	2024	Transforms documents so the model can better distinguish valid from adversarial instructions	✓	ASR: 26.3 → 1 (gpt-3.5-turbo)	N/A	N/A
IsolateGPT [35]	2025	Introduces an isolation system for LLMs through structured agent communication protocols	✓	ASR : 20.2 → 7.6 (gpt-4)	InjectAgent [234]	[235]
CaMeL [38]	2025	Prevents data and control-flow hijacking with a dual-LLM architecture with a safety execution layer	✓	ASR: 9.3 → 0 (avg: claude, gemini, gpt, o3, o4)	AgentDojo [157]	[236]
SafetyRAG [137]	2025	Modular framework that verifies and generates modules to attacks	✓	ASR: 65 → 35 (gpt-3.5-turbo), 86 → 7 (llama-2-13b)	Own	N/A
Instruction Hierarchy [68]	2024	Fine-tuning with synthetic data ro enforce instruction hierarchy: system > user > tool	✗	ASR: 37 → 8 (gpt-3.5-turbo)	N/A	N/A
ISE [63]	2024	Incorporates hierarchy information into embeddings to prioritize instructions	✗	ASR: 49.2 → 33.7 (llama)	AlpacaEval [195]	N/A

Table A.11: Overview of evaluation-time interventions.

Ref.	Year	Description	Model-agnostic	Source code
[141]	2025	Evaluates the impact of jailbreak defense strategies on LLMs, measuring their utility, safety, and usability through automated benchmarks	✓	N/A
PromptShield [139]	2025	Benchmarks prompt injection classifiers	✓	[237]
[140]	2025	Quantitatively measures an LLM's ability to separate instructions from data	✓	N/A
[142]	2025	Post-hoc confidence calibration of LLM-based guard models via Temperature Scaling and Contextual Calibration	✓	[238]
JailbreakBench [143]	2024	Benchmarks LLMs for robustness based on a 100 policy-violating behaviors and adversarial prompts	✓	[239]
Garak [144]	2024	A security-probing framework that evaluates LLM vulnerabilities by automatically generating and executing targeted adversarial probes	✓	[240]
AutoDAN [47]	2024	Evolves stealthy jailbreak prompts able to bypass aligned LLM safeguards based on genetic algorithm	✓	[241]
JailbreakHunter [138]	2024	Identifies jailbreak prompts in large-scale human-LLM conversational datasets and provides visual analytics	✓	N/A
GCG/Advbench [19]	2023	Produces adversarial prompts using the gradient-based suffix search method	✓	[242]
PAIR [26]	2023	PAIR utilizes an attacker LLM to continuously refine prompts based on target model responses, achieving effective black-box jailbreaks in fewer than 20 queries	✓	[243]